Language Design



So, you want to design your own language? Of course you do. Or perhaps you are taking a class and are being forced to create a programming language under penalty of a bad grade. What kinds of things do you need to know?

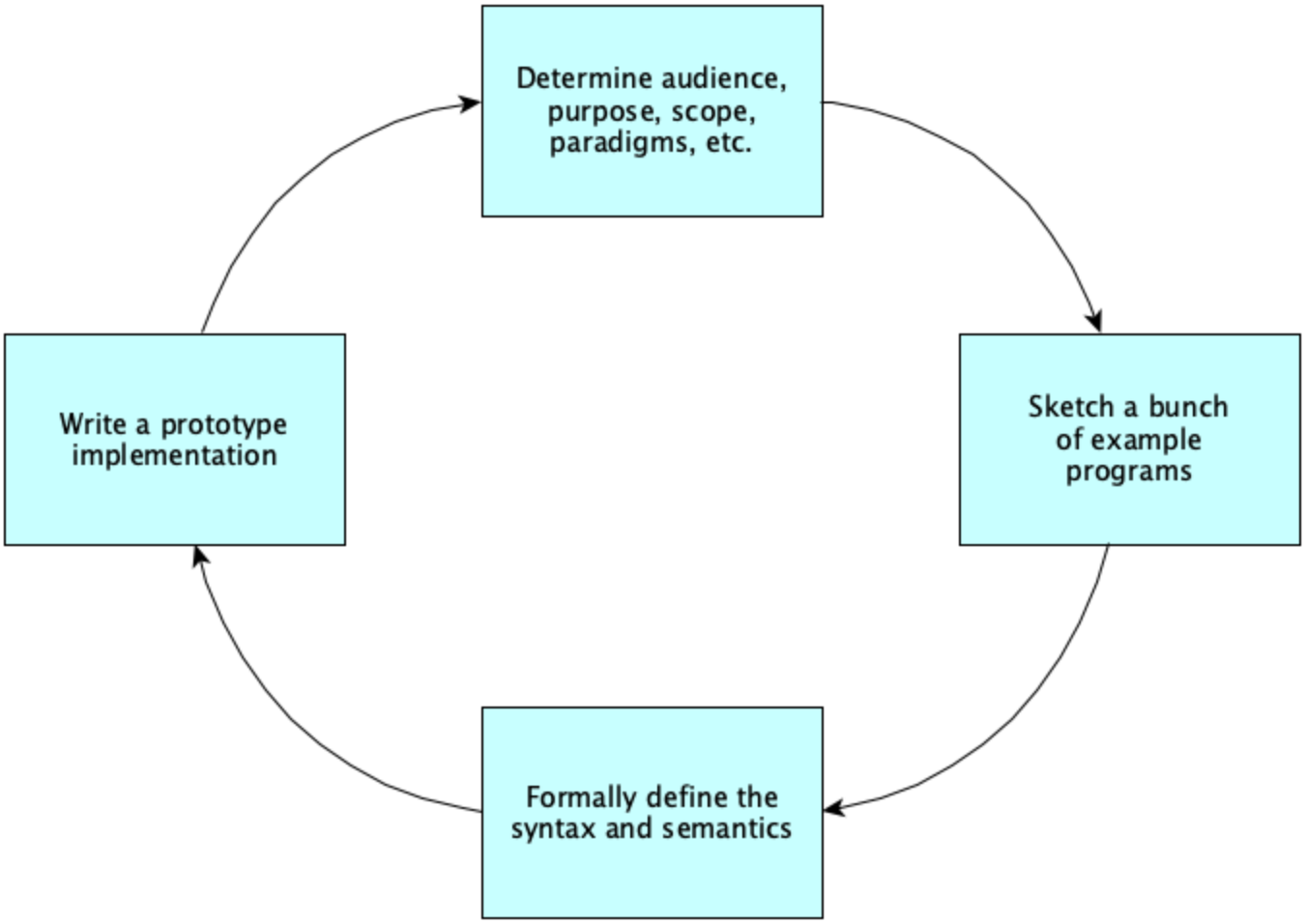
CONTENTS

Designing a Language • Prerequisites • Getting Ready • Getting Started • Choosing a Starter Set of Features • From Features To Abstract Syntax • From Abstract Syntax to Concrete Syntax • Defining Your Language • Prototyping • Examples • More Examples • Recall Practice • Summary

Designing a Language

Of course you want to design (and implement!) your own programming language! It's fun. It's creative. It's empowering.

How do we do it? In a nutshell, the process is iterative, cycling between four phases:



Doing the phases over and over is important; for example, while writing the compiler, you may be like “woah this is impossible” and then realize “oh shoot this part of the language wasn’t designed right!”

Prerequisites

It helps to be experienced. If you’re not, that’s okay, actually—you might get lucky!

But don’t mistake creativity for luck that shows up without pre-existing knowledge. The most creative people are those with a *lot* of knowledge and experience. So you should still study and practice!

Your success as a language designer will be massively aided by knowledge in three main areas:

1. **Programming Paradigms**. You should know a number of different ways that computations can be structured. These include: imperative | declarative | structured | object-oriented

functional | applicative | concatenative | logic | protocol-oriented | aspect-oriented | array | event-driven | dataflow | agent-based etc.

See Wikipedia's list of programming paradigms.

2. **Programming Language Concepts**. You should have a good sense of most, if not all, of the following things: Sequencing, conditional execution, iteration, recursion, functional decomposition, modularity, synchronization, metaprogramming, binding, scope, extent, volatility, subtyping, pattern matching, type inference, closures, prototypes, introspection, instrumentation, annotations, decorators, memoization, traits, streams, monads, actors, mailboxes, comprehensions, continuations, wildcards, promises, regular expressions, proxies, transactional memory, inheritance, polymorphism, parameter modes, type classes, generics, reflection, concurrency, parallelism, distribution, persistence, transactions, garbage collection, and many more terms.

   I'm working on a glossary of such terms that may be helpful to review.

3. **Existing ProgrammingLanguages**. It's nice to have a feel for a variety of languages. Here are a few that are good to be familiar with (and the reasons why they are good to study):

   - **Python**, for basic imperative programming and scripting
   - **Smalltalk**, for OOP done beautifully
   - **JavaScript**, for event-driven features, async, and promises
   - **Io**, for ideas on building up everything from a minimal semantics
   - **Julia**, for multiple dispatch
   - **Clojure**, **Racket**, **Scheme**, and **Common Lisp**, for Lispiness (macros, etc.)
   - **Standard ML**, **OCaml**, **F#**, and **Elm**, for Hindley-Milner typing and more
   - **Haskell**, for typeclasses and functional purity
   - **eToys**, **Scratch**, **Snap!**, and **GP** for the blocks approach
   - **Java** and **C#**, for being enterprisey
   - **Kotlin**, **Ceylon**, and **Scala**, as examples of evolving Java
   - **Erlang** and **Elixir**, for expressing concurrent processes in a distributed fashion
   - **Fortran**, **Chapel**, and **Parasail**, for expressing muticore and parallel programming
   - **PureScript**, **TypeScript**, and **ClojureScript**, as examples of evolving JavaScript
   - **C**, because it is the quintessential systems language
   - **C++**, **Rust**, **Zig**, and **Odin** for making systems programming less painful and more secure
   - **Go** and **Swift**, for more examples of modern, but generally mainline, ideas

- **J** and **K**, for array programming
- **Idris**, for dependent typing
- **Prolog** and **Mercury**, for logic programming
- **Forth** and **Factor**, for concatenative programming
- **Quipper**, because it's a language for quantum computing
- **Brainfuck**, **Malbolge**, **LOLCODE**, **Whitespace**, and other classic esoterics
- **GolfScript**, **CJam**, **Pyth**, **Jelly**, and other golfing languages, because this gives you a good sense of putting power into small syntactic constructs
- **Piet** and **Hexagony**, for, well, check them out....

Also check out this mini-encyclopedia of 70 languages.

Here are some excellent cross-language comparisons that help you to hone your understanding of how different syntaxes can express the same ideas:

- Syntax Across Languages
- Wikipedia Programming Language Comparisons
- Hyperpolyglot

These are really good too:

- Showcase of Languages
- Languages Through the Years

Getting Ready

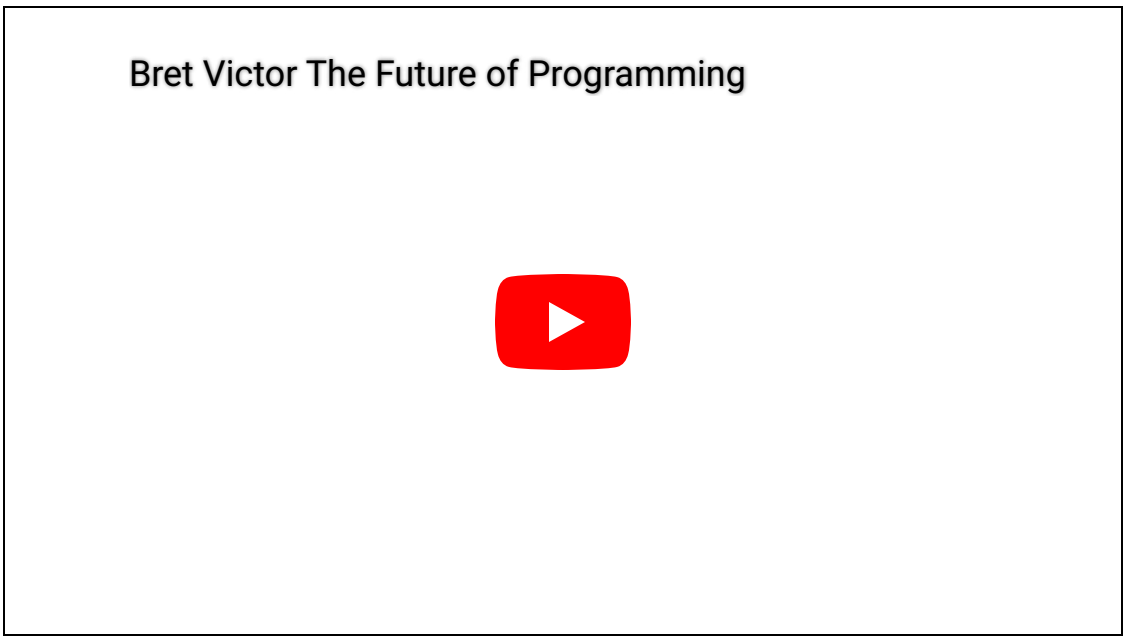
Remember, many people have designed languages before you. They made mistakes. They came up with brilliant ideas. Many were wildly successful. Some never made it big. Some people have brought in years of research on how people think and learn to come up with principles for language (and environment) design.

You should learn form their experiences.

Study classic papers. Read web essays. Visit online courses. Here is a small sampling of things to study and places to look for more information:

- Hints on Programming Language Design by Tony Hoare
- Learnable Programming by Bret Victor
- Five Questions about Language Design by Paul Graham
- For early classic papers, check out the massive bibliography of this paper
- Also look at the papers from Kathleen Fisher's Programming Language Design class

Think about the future:



And understand that traditional, mainstream programming languages, are not at all the epitome of computational expression. Languages can be much more:



Getting Started

Ready to strike out on your own? Here are some things to think about, in the form of a, you guessed it, a checklist:

- ✅ Do you have a specific audience in mind? Artists? Graphic designers? AI researchers? Numeric and Scientific nerds? Natural language types? Game developers? Machine learning people? Animators? High performance folks? System programmers? Or do you want a general purpose language? Or do you just want to do what *you* want?
- ✅ Understand the audience that the language is designed for, and what kinds of things they want to create with it (or problems they want to solve with it).
- ✅ Determine if your language is to be (1) a reasonable, usable language or (2) an esoteric/joke/golfing language.
- ✅ Determine if it is to be pragmatic, idealistic, researchy, or evil.
- ✅ Determine whether you want your language firmly in one camp—OO, functional, logic, concatenative, plain-imperative—or be a multiparadigm symphony. Or a multiparadigm cacophony.
- ✅ Determine whether it is to be built on a single characteristic building block (a *crystallization of style*) or one with a huge variety of syntactic forms for multiple semantic aspects (an *agglutination of features*).
- ✅ Determine your concurrency model. Do you want all your programs to be single-threaded and sequential? Or are you looking for something event-driven and async? Or multithreaded? Or distributed?

Choosing a Starter Set of Features

Come up with a list of capabilities, or features. Make sure they enable programmers to express their creations by following the suggestions and principles in the Learnable Programming essay, including:

- Make meaning transparent
- Explain in context
- Make flow and time tangible and visible
- Show the data, and avoid hidden state
- Use metaphor
- Allow easy decomposition and recomposition
- Allow the programmer to write readable code—you *do* support mentioning parameter names in calls, right?

***Exercise**: Skim the Learnable Programming essay. Or, better, read the whole thing ~~if~~ when you have time.*

What kind of questions might you have here? Here are some totally random ideas:

- If your language supports functions:
  - Must you pass the exact number of arguments the function expects?
  - Can you specify parameter names in the call?
  - Do you have rest parameters? optional parameters? keyword parameters? default parameters?
  - Are the parameters typed?
  - Can parameters be modified? Are they passed by value, value-result, reference, name, etc?
  - Are arguments evaluated L-R, R-L, in parallel, or in arbitrary order?
  - How does a function return a value? Can it return zero or more values or just one?
  - Are functions first-class? If so, do you use deep binding or shallow binding?
  - Is recursion allowed?
  - Can you test functions for equality?
  - Can functions be anonymous?
- What is your type system like?
  - Static vs. dynamic, strong vs. weak, manifest vs. implicit?
  - Do you distinguish primitive types from reference types?
  - Do you have all those different numeric types based on bit size? What about decimals and ratios?
  - Do you have a separate character type? A separate boolean type?
  - Can you get the type of an expression at runtime?
  - Are types objects?
  - Do you have supertypes and subtypes? Multiple-inheritance? If so, how do you handle name collisions?
  - Are types the same as classes? Can you add new types?
  - Do you have both sum and product types?
  - Do functions have a type?
  - Are there pointer types?
  - Are there parameterized types? If so, are they (mostly) covariant, contravariant, or invariant?
  - Any dependent types?
- What do your expressions look like?
  - And does your runtime follow eager or lazy evaluation? Or both?
  - Do you have only prefix, only postfix, only infix, or a wild mix of operators?
  - Can you overload operators? If so, how?

    - Can you change the precedence of operators? Even the built-in ones? At runtime?
    - How much type inference do you have?
    - Can you mark variables as mutable or immutable?
    - Are your variables bound or assigned? Can they be assigned more than once?
    - Do you have destructuring and/or pattern matching?
    - How do you determine scope? Do you implicitly or explicitly import into inner scopes?
    - Are there keywords that define access like `public`, `private`, and `protected`, or are there conventions, like in Go, where capitalized entities are implicitly exportable and lower-cased entities are private?
    - Is there a `let`-expression for super-local scopes?
    - Is shadowing allowed?
    - Do you have anything like JavaScript's `this` expression?
- How do you express control flow?
    - Do you like expression-orientation or do you have real statements?
    - How do you express sequential flow vs. nondeterministic flow vs. parallel flow?
    - If you have nondeterminism, how do you ensure fairness?
    - Must guards in a multiway selection be executed in any particular order?
    - Do you have short-circuit operators? Iterator objects? Generators?
    - Do you have all the loops: (1) forever, (2) n times, (3) while/until a condition, (4) through a numeric range (with an optional step size), or (5) through a collection?
    - Do you have `break` and `continue`? Anything like Ruby's `retry` and `redo`?
    - Do you have exceptions? Or do you need Go-style constructs? Or do you have nullables everywhere?
    - Do you have a timer for sleeping, delaying execution, or running on intervals?
    - Can I haz `goto`?
- How do you support concurrency?
    - Threads, or events?
    - Shared memory only? Message passing only? Or both? If shared memory, is it mutable? If so, do you have locks, higher-level synchronization devices, or some kind of transactional memory?
    - Do you support different levels of granularity of concurrency?
    - Are tasks spawned implicitly when their enclosing block begins, or do they require explicit invocation?
    - Do tasks die when the enclosing block or spawning task terminates? Or does the spawner wait for all internal tasks to terminate?
    - Do taks know who started them?
    - If an asynchronous task is launched, how is a result obtained? Callback, promise, or some other mechanism?

    - Is message passing done via named channels, named processes, or bulletin board?
    - Is message passing always synchronous, asynchronous, or both?
    - Is there timed or conditional synchronization mechanisms?
    - Can tasks detect their own state?
- How meta are you?
    - Can a program get a list of its global variables, loaded classes, top-level functions, etc.? Can a class get a list of its fields, methods, constructors, superclass(es), subclasses, etc.? Can a function get its parameters, locals, return types, etc.?
    - Can a function find out who called it?
    - Can a variable be read or set via its name (a string) only? Can a function or method be invoked by its name (a string) only? Can new variables or types or functions be created at run time, given only their name and some indication of how they should look?
    - Does the language have a macro system? If so, is it string-based or AST-based?
    - Is it possible to unquote within a macro?
    - Are macros hygienic? What syntactic devices are provided to explicitly capture in- scope variables, if any?

> ***What did I just read?***
>
> *Feeling like only about 20% of the questions above made any sense? Feeling like that vocabulary came out of nowhere? That's fine for now. Learning about programming languages can be a never-ending lifelong journey, but you can use the questions you don't quite understand now as a place to start some research.*
>
> *Oh, I have a glossary you might find helpful.*

From Features To Abstract Syntax

When you have a good idea of your language features, you'll want to figure out a good way to organize them, structurally. This is known as your language's **abstract syntax**. In an abstract syntax we don't worry much about punctuation and parentheses and such microscopic details. We are interested in the overall structure. Here's a look at abstract syntax trees in JavaScript:



When defining your language, you will want to specify exactly what the AST Nodes are, how they are related to each other, and what their properties are. For JavaScript, the AST Node types come from a specification called EsTree. I've summarized the main interfaces here:

```
Program sourceType:["script"|"module"] body:[Statement|ModuleDeclaration]
Statement
    Declaration
        FunctionDeclaration(Function) id:Identifier
        VariableDeclaration declarations:[VariableDeclarator] kind:("var"|"let"|"
        ClassDeclaration(Class) id:Identifier
    EmptyStatement
    DebuggerStatement
    ExpressionStatement expression:Expression
    BlockStatement body:[Statement]
    ReturnStatement argument:Expression?
    LabeledStatement label:Identifier body:Statement
    BreakStatement label:Identifier?
    ContinueStatement label:Identifier?
    IfStatement test:Expression consequent:Statement alternate:Statement?
    SwitchStatement discriminant:Expression cases:[SwitchCase]
    WhileStatement test:Expression body:Statement
    DoWhileStatement body:Statement test:Expression
    ForStatement init:(VariableDeclaration|Expression)? test:Expression? update:E
    ForInStatement left:(VariableDeclaration|Pattern) right:Expression body:State
        ForOfStatement await:boolean
    ThrowStatement argument:Expression
```

```
    TryStatement block:BlockStatement handler:CatchClause? finalizer:BlockStateme
    WithStatement object:Expression body:Statement
Function id:Identifier? params:[Pattern] body:BlockStatement generator:bool async
VariableDeclarator id:Pattern init:Expression?
SwitchCase test:Expression? consequent:[Statement]
CatchClause param:(Pattern?) body:BlockStatement
Expression
    ThisExpression
    Identifier(Pattern) name:string
    Literal value:(string|bool|number|Regexp|bigint)?
        RegExpLiteral regex:{pattern:string flags:string}
        BigIntLiteral bigint:string
    ArrayExpression elements:[(Expression|SpreadElement)?]
    ObjectExpression properties:[Property|SpreadElement]
    FunctionExpression(Function)
    ArrowFunctionExpression(Function) body:(BlockStatement|Expression) expression
    UnaryExpression operator:UnaryOperator prefix:bool argument:Expression
    UpdateExpression operator:UpdateOperator argument:expression prefix:bool
    BinaryExpression operator:BinaryOperator left:Expression right:Expression
    AssignmentExpression operator:AssignmentOperator left:Pattern right:Expressio
    LogicalExpression operator:LogicalOperator left:Expression right:Expression
    MemberExpression(ChainElement) object:(Expression|Super) property:Expression
    ChainExpression expression:ChainElement
    ConditionalExpression test:Expression consequent:Expression alternate:Express
    CallExpression(ChainElement) callee:(Expression|Super) arguments:[(Expression
    YieldExpression argument:Expression? delegate:bool
    TemplateLiteral quasis:[TemplateElement] expressions:[Expression]
    TaggedTemplateExpression tag:Expression quasi:TemplateLiteral
    NewExpression
    SequenceExpression expressions:[Expression]
    ClassExpression(Class)
    AwaitExpression argument:Expression
    ImportExpression source:Expression
    MetaProperty meta:Identifier property:Identifier
Class id:Identifier? superClass:Expression? body:ClassBody
ClassBody body:[MethodDefinition]
MethodDefinition key:Expression value:FunctionExpression kind:("constructor"|"met
SpreadElement argument:Expression
Property key:Expression value:Expression kind:("init"|"get"|"set") method:bool sh
```

```
    AssignmentProperty value:Pattern kind:"init" method:false
Pattern
    ObjectPattern properties:[AssignmentProperty|RestElement]
    ArrayPattern elements:[Pattern?]
    RestElement argument:Pattern
    AssignmentPattern left:Pattern right:Expression
Super
TemplateElement tail:boolean value:{cooked:string? raw:string}
ChainElement optional:boolean
enum UnaryOperator {"-"|"+"|"!"|"~"|"typeof"|"void"|"delete"}
enum UpdateOperator {"++"|"--"}
enum BinaryOperator {"=="|"!="|"==="|"!=="|"<"|"<="|">"|">="|"<<"|">>"|">>>"|"+"|
enum AssignmentOperator {"="|"+="|"-="|"*="|"/="|"%="|"**="|"<<="|">>="|">>>="|"|
enum LogicalOperator {"||"|"&&"|"??"}
ModuleDeclaration
    ImportDeclaration specifiers:[ImportSpecifier|ImportDefaultSpecifier|ImportNa
    ExportNamedDeclaration declaration:Declaration? specifiers:[ExportSpecifier]
    ExportDefaultDeclaration declaration:(Declaration|Expression)
    ExportAllDeclaration source:Literal exported:(Identifier?)
ModuleSpecifier local:Identifier
    ImportSpecifier imported:Identifier
    ImportDefaultSpecifier
    ImportNamespaceSpecifier
    ExportSpecifier exported:Identifier
```

I've built an interactive application for you to explore AST generation. Please try it out!.

***Exercise**: Please try it out.*

> ***Too much detail?***
>
> *You may notice that the EsTree specification has a lot more detail than the hand-drawn ASTs in the video above. This is okay. The specification is used to build actual compilers and interpreters, while for hand-drawn ASTs we just like to give the big picture and can take some "shortcuts."*

From Abstract Syntax to Concrete Syntax

Now it's time to think about what your language will really look like! Remember that design is creative and iterative, so you will want to begin, like all artists do, with sketches:

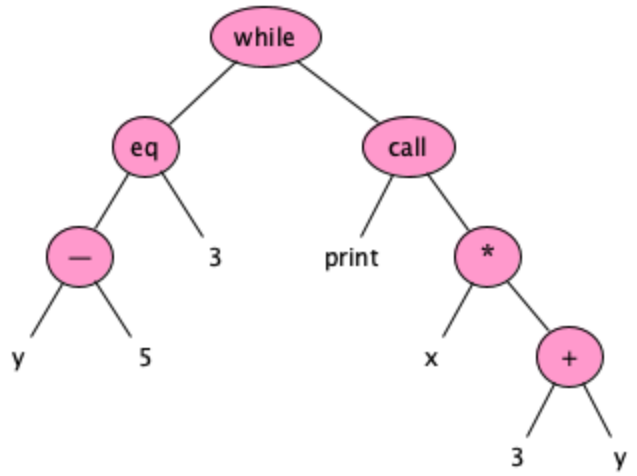
- Sketch programs or program fragments in your language.
- Sketch fragments that are *not* in your language.
- Come up with a nice, consistent, syntactic theme.

Do a lot of experimentation here! You will probably want to put creative effort into designing languages people like to use! What kind of syntax issues do they deal with? Dozens, actually, and we can't cover them all. But how about a taste of just a few. We'll peek at just a few issues that sometimes generate strong opinions.

Overall Phrase Structure

You will need to adopt a scheme for showing structure. The popular approaches are: Curly-brace (JavaScript, Java, C++, C#), Terminal-end (Ruby, Ada), Nested parentheses (Lisp, Clojure, Racket), Indentation (Python), Blocks (EToys, Scratch, Snap!), Pictures (Piet), Other (Haskell, Erlang, Prolog).

The important idea here is that a single abstract syntax can be realized with many different **concrete syntaxes**. A concrete syntax specifies exactly which strings of characters make up structurally valid programs. For example, the AST:



represents each of the following (and more!):

```
while y - 5 == 3:
    print(x * (3 + y))
```

```
while y - 5 == 3 {
  print(x * (3 + y))
}
```

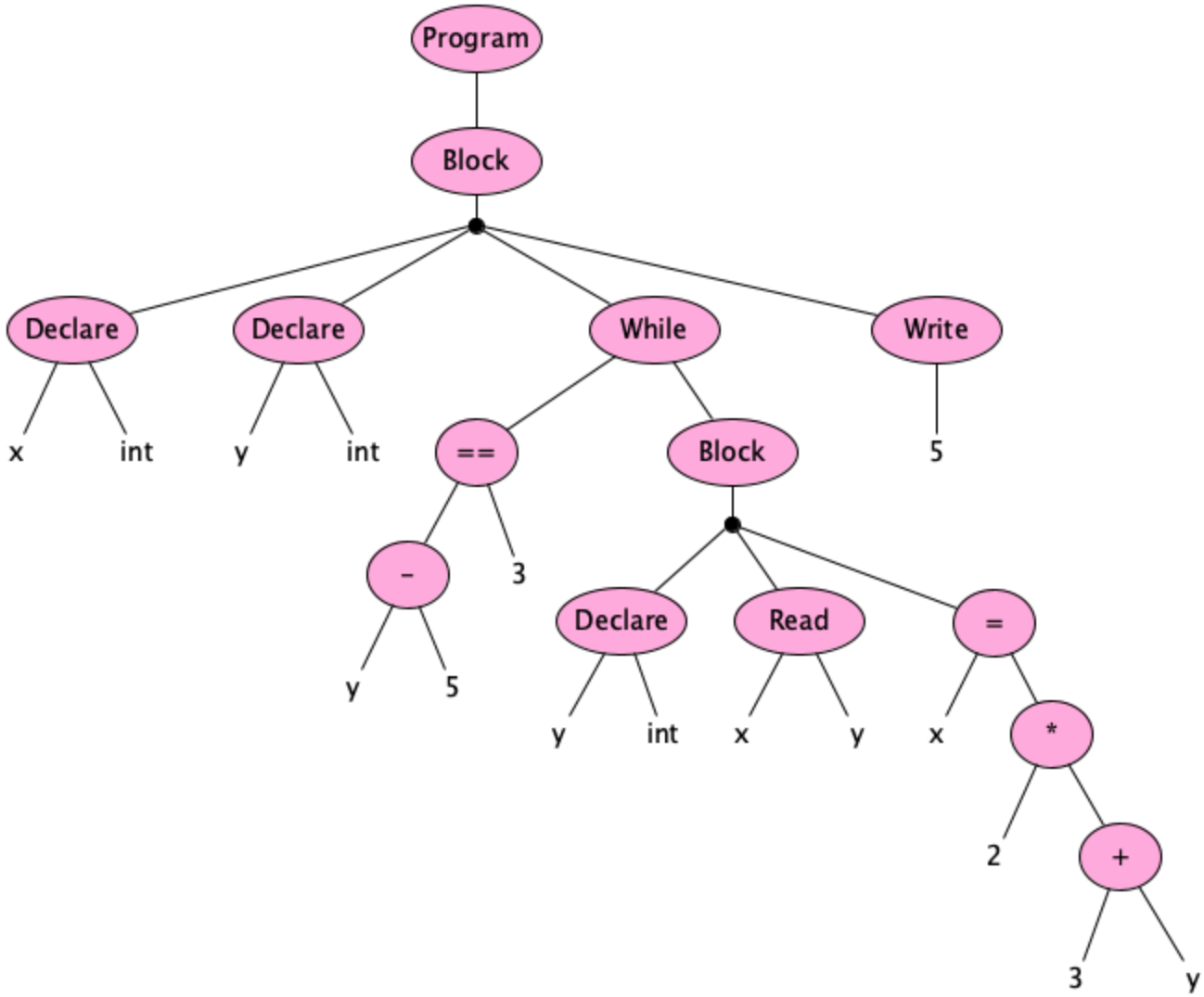
```
while y - 5 == 3 loop
  print(x * (3 + y))
end
```

```
(while (= (- y 5) 3)
    (print (* x (+ 3 y))))
```

> ***Exercise**: The last syntax is used in Clojure, Lisp, Scheme and others. Note that in some ways, this **IS** the abstract syntax. How so?*

> ***Exercise**: Here is another:* `y 5 - 3 == [x 3 y + * print] while`. *Do you know of, or can you find, any languages which have that kind of syntax?*

In addition to structure, your choice of keywords, operators, punctuation (or lack thereof) are part of your design. Here's an abstract syntax:



Let's try out a few things:

```
program:
    var x, y: integer
    while y - 5 == 3:
        var y: integer
        get(x)
        get(y)
        x = 2 * (3 + y)
    put(5)
```

```
int x, y;
while y - 5 = 3 {
    int y;
    STDIN -> x;
    STDIN -> y;
```

```
    x <- 2 * (3 + y);
}
STDOUT <- 5;
```

```
COMMENT THIS LOOKS LIKE OLD CODE
DECLARE INT X.
DECLARE INT Y.
WHILE DIFFERENCE OF Y AND 5 IS 3 LOOP:
    DECLARE INT Y.
    READ FROM STDIN INTO X.
    READ FROM STDIN INTO Y.
    MOVE PRODUCT OF 2 AND (SUM OF 3 AND Y) INTO X.
END LOOP.
WRITE 5 TO STDOUT.
```

```
(program
  (declare x int)
  (declare y int)
  (while (= (- y 5) 3)
    (define (y int))
    (read x y)
    (assign x (* 2 (+ 3 y)))
  )
  (write 5)
)
```

***Exercise**: Play around and sketch a few more.*

Delimiters

How to separate one construct from another is a really big issue in syntax design, believe it or not. We can identify two main classes of languages: those in which newlines are significant and those in which they are not.

"Insignificant" Newlines

In many languages, newlines are just like any other whitespace character (except for minor exceptions such as single-line comments and single-line string literals. Then, unless you have an S-Expression-based syntax as in LISP, Scheme, and Clojure, you'll need semicolons to terminate (or separate) statements. This means you *can* (but shouldn't) write code like:



```c
#define ZERO 0
    unsigned  gcd(   unsigned   int  // Euclid's algorithm
      x,unsigned   y) {   while ( /* hello */  x>   ZERO
   ){unsigned temp=x;x=y   %x;y  = temp ;}return

   y ;}
```

"Significant" Newlines

Where you place your newlines matters greatly in, let's see, Assembly languages, Python, Ruby, JavaScript, Elm, Haskell, Go, Swift, and yes, many others. The rules can get pretty technical.

Python scripts are defined as sequences of **logical lines**, delimited by the token NEWLINE. A statement may not cross logical lines, except in the case of **compound statements** in which each constituent **simple statement** ends with a NEWLINE. Logical lines are made up of one or more physical lines according to line joining rules. Lines are *implicitly* jointed within parentheses, brackets, or braces; lines can be *explicitly* joined by ending with a backslash. These rules are somewhat exclusive of comments and string literals.

Ruby looks at the end of each line and says "well if up to here it looks like we've completed a statement the we have." This means you have to be careful where you break lines:



```ruby
puts 5
  + 3
puts 5 +
  3
```

prints 5 then 8.

> ***Exercise**: Why?*

"Possibly Significant" Newlines

JavaScript requires most statements to be terminated by semicolons, but the *compiler will put one in for you if it looks like you might have missed one*. The rules by which this **automatic semicolon insertion** (ASI) is done have to be learned and they might be hard to remember.



If you are going to be a serious JavaScript programmer, you need to learning the rules of ASI **whether you choose to use semicolons or not**.

> ***Exercise**: Research the famous Rules of Automatic Semicolon Insertion. Which statements are supposed to be terminated by a semicolon? When is a semicolon inserted? Give four examples of how writing JavaScript in a "free-form" manner is impossible because of semicolon insertion.*

> ***Exercise**: Get your ASI Certification*

Some people feel very strongly whether to use or not to use semicolons:



douglascrockford commented — a year ago

That is insanely stupid code. I am not going to dumb down JSMin for this case.



douglascrockford commented — a year ago

TC39 is considering the use of ! as an infix operator. This code will break in the future. Fix it now. Learn to use semicolons properly. ! is not intended to be a statement separator. ; is.



fat commented — a year ago

i have learned to use them, that's why there isn't one present.



kitcambridge commented — a year ago

> i have learned to use them, that's why there isn't one present.

Zzzzing!

Function Calls

Most programming languages have functions. Seriously. But there are a lot of ways to work them into your design. Basic questions include: Must functions have exactly one argument, or zero or more arguments? Parens or no parens? Positional or keyword arguments? Argument labels? If no arguments, can we omit parentheses?

You can play around and see what you can come up with:

- `push(myStack, 55)`
- `push myStack 55`
- `[push myStack 55]`
- `(push myStack 55)`
- `push(on: myStack, theValue: 55)`
- `push(theValue: 55, on: myStack)`
- `push on:myStack theValue:55`
- `push({ on: myStack, theValue: 55 })`
- `push { on: myStack, theValue: 55 }`
- `push({ theValue: 55, on: myStack })`

You might want to consider an ultra-low precedence function application, like they have in Haskell and F#:

- `sum (filter even (map square a))`
- `sum $ filter even $ map square $ a`
- `sum <| filter even <| map square <| a`
- `a |> filter even |> map square |> sum`

The flip side of function calls is function definitions. You're likely familiar with default parameters, and rest parameters. Python has cool mechanisms for requiring arguments to be positional or keyword, based on the definition. Examples:

- `def sqrt(x, /)`
- `def line(*, x1, x2, y1, y2, width, style, color)`
- `def f(a, b, /, c, d, *, e, f)`

Syntactic Sugar

Syntactic sugar refers to forms in a language that make certain things easier to express, but can be considered surface translations of more basic forms.

This is best understood by example. There are zillions of examples out there. Here are a few. (**Disclaimer**: Some of these are just examples I made up and are not part of any real language.)

Construct	Desugared Form	Description
`x += n`	`x = x + n`	Compound assignment
`a + b`	`operator+(a, b)` *or* `"+"(a, b)` *or* `__add__(a, b)`	Common in languages that allow overloading
`a[i]`	`*(a + i)`	(C, C++ pointer arithmetic) And `i[a]` works too!
`p -> x`	`(*p).x`	(C, C++) Field of struct being pointed to
`f`	`f()`	Some languages let you leave off parentheses in calls with no arguments
`f x`	`f(x)` *or* `x.f()`	Some languages let you leave off parentheses in calls with one argument

`x op y`	`op(x, y)` *or* `x.op(y)`	Some languages let you leave off parentheses in calls with two arguments
`let x=E1 in E2`	`(x => E2)(E1)`	Let-expression (in functional languages)
`(E1 ; E2)`	`(() => E2)(E1)`	Expression sequencing (in eager functional languages)

`r = [s for x in a if e]`	`r = []`
	`for x in a:`
	`  if e:`
	`  r.add(s)`
		List comprehension

`x orelse y`	`if x then x else y`	(Standard ML) short-circuit disjunction
`x andalso y`	`if x then y else x`	(Standard ML) short-circuit conjunction
`[x, y, z]`	`x :: y :: z :: nil`	Lists in Standard ML
`"a${x}b"`	`"a" + x + "b"`	String interpolation

***Exercise**: Find some more examples.*

When the sugared form is completely gratuitous or actually makes the code *less* readable, you sometimes hear the term **syntactic syrup** or **syntactic saccharin**.

Syntactic Salt

Here's the definition from The New Hacker's Dictionary:

> *The opposite of syntactic sugar, a feature designed to make it harder to write bad code. Specifically, syntactic salt is a hoop the programmer must jump through just to prove that he knows what's going on, rather than to express a program action. Some programmers consider required type declarations to be syntactic salt. A requirement to write "*`end if`*", "*`end while`*", "*`end do`*", etc. to terminate the last block controlled by a control construct (as opposed to just "*`end`*") would definitely be syntactic salt. Syntactic salt is like the real thing in that it tends to raise hackers' blood pressures in an unhealthy way.*

Candygrammars

Some people love verbose code, because explicit is better than implicit. But if you are language designer, be pragmatic: there is such a thing as code that is too verbose. What about trying to make the code like human language? Here's an example in Hypertalk (taken from Wikipedia):

```
on mouseDown
  answer file "Please select a text file to open."
  if it is empty then exit mouseDown
  put it into filePath
  if there is a file filePath then
    open file filePath
    read from file filePath until return
    put it into cd fld "some field"
    close file filePath
    set the textStyle of character 1 to 10 of card field "some field" to bold
  end if
end mouseDown
```

And here's an example from a language that some students developed, and regretted:

```
to get the truth value prime of whole number n:
    return no if n < 2
    for each d in 3 to n - 1 by 2:
        return no if d divides n
    end
    return yes
end
for each k in 1 to 100:
    write k if prime(k)
end
```

In practice this kind of verbosity is worse than it sounds. Here's what the New Hacker's Dictionary has to say about this:

> ***candygrammar /n./*** *A programming-language grammar that is mostly syntactic sugar; the term is also a play on "candygram." COBOL, Apple's Hypertalk language, and a lot of the so-called "4GL" database languages share this property. The usual intent of such designs is that they be as English-like as possible, on the theory that they will then be easier for unskilled people to program. This intention comes to grief on the reality that syntax isn't what makes programming hard; it's the mental effort and*

organization required to specify an algorithm precisely that costs. Thus the invariable result is that candygrammar languages are just as difficult to program in as terser ones, and far more painful for the experienced hacker.

[The overtones from the old Chevy Chase skit on Saturday Night Live should not be overlooked. This was a "Jaws" parody. Someone lurking outside an apartment door tries all kinds of bogus ways to get the occupant to open up, while ominous music plays in the background. The last attempt is a half-hearted "Candygram!" When the door is opened, a shark bursts in and chomps the poor occupant. There is a moral here for those attracted to candygrammars.]

Terseness

Some languages pride themselves on doing a whole lot with few characters:

An example from Ruby (do you see what this does?):



```ruby
c = Hash.new 0
ARGF.each {|l| l.scan(/[A-Z']+/i).map {|w| c[w.downcase] += 1}}
c.keys.sort.each {|w| puts "#{w}, #{c[w]}"}
```

An example from APL (The 99 bottles of beer program taken from Rosetta Code):



```
bob  ←  { (⍕⍵), ' bottle', (1=⍵)↓'s of beer'}
bobw ←  {(bob ⍵) , ' on the wall'}
beer ←  { (bobw ⍵) , ', ', (bob ⍵) , '; take one down and pass it around, ', bobw
↑beer¨ ⌽(1-⎕IO)+⍳99
```

Here's APL again, with an expression to find all the prime numbers up to R:



```
(~R∈R∘.×R)/R←1↓⍳R
```

Some people love terse, concise code, because it says only what it needs and reduces the cognitive load, leaving you with less useless noisy syntax to learn. But if you are language designer, be pragmatic: there is such a thing as code that is too terse. Unless...that's your goal....

Golfing Languages

Golfing languages take terseness to the next level. A golfing language is a kind of esoteric programming language (a non-practical language created to experiment with weird ideas, be hard to program in, or be humorous) that allows programs to be written in an insanely small number of characters (or bytes).

Here are some CJam programs:

- `"Hello, world!"`
- `5{"Hello, world"oNo}*`
- `0X{_2$+}A*]N*`
- `l~@-@@-mh`
- `1{_B<}{_'**N+o)}w;`

Here are some Pyth programs (taken from the documentation):

- `"Hello, world!`
- `FNrZhTN`
- `FNUhTN`
- `VhTN`
- `K1FNr1hQ=K*KN;K`
- `.!Q`
- `WtQ=Q?%Q2h*Q3/Q2Q`
- `A(Z1;VhhTGA(H+GH`

> ***Exercise**: Find a bunch more examples of CJam and Pyth programs. Try them out. You can run them both at TIO. For Pyth, there's also a hosted interpreter with a Cheat Sheet.*

> ***Exercise**: Try out Stax.*

Defining Your Language

Traditionally, real world language definitions come in three main flavors:

- An official document, with a mix of formal notation and informal descriptions (Very common)
- An official document, with 100% of the definition specified in a formal notation (Very rare)
- A "reference implementation," namely a compiler or interpreter, so that the language "definition" is simply "whatever this program does" (Typical for some scripting languages). An advantage if this approach is that there are never any compiler bugs!

> ***Exercise**: Why exactly is a reference implementation compiler by definition bug-free? If being bug-free is so great, why don't all languages do this? What are the downsides?*

An official definition will have three parts:

Syntax (Structure)	**Semantics (Meaning)**	
	Statics	**Dynamics**
What are the structural entities (e.g., declarations, expressions, statements, modules) and how do they fit together, perhaps with punctuation?	What are the non-structural rules that define a legal program (e.g., type checks, argument-parameter matching rules, visibility rules, etc.)?	What does a program do? What effects do each of the forms of a well-structured, legal program have on the run-time environment?

Why are there three parts instead of two (i.e., just syntax and semantics)? Here's why. While everyone might agree that the following is *structurally* malformed:

```
#<include > stdio.h
main() int }
    printf["Hello, world!\n");]
{
```

the following program looks good in terms of "structure" but it's actually *meaningless* since it violates a **contextual rule** that says identifiers must be declared before use:

```
int main() {
    printf("%d\n", x);
}
```

We say the latter program has **static semantic errors** because they can be detected by a compiler before the program is ever run. This is in contrast to a dynamic semantic error, which can only be

detected at run time.

> **Example**: You've probably heard the distinction between "static" and "dynamic" before. Perhaps you know that "static typing" involves type checking is done *prior to* program execution and "dynamic typing" involves checking *during* program execution. Most languages do a little of both, but one or the other usually predominates. Sometimes you get a good deal of both: in TypeScript for example, you have a set of static types which is much larger and completely different than the eight dynamic types. Fun.

Defining the Syntax

Let's see how we would formally specify the syntax for the simple language Astro. Assume we've gone through the first two design phases, and we've sketched out a program that shows all the features:

```
// A simple program in Astro

radius = 55.2 * (-cos(2.8E-20) + 89) % 21;    // assignment statement
the_area = π * radius ** 2;                   // a built-in identifier
print hypot(2.28, 3 - radius) / the_area;     // print statement
```

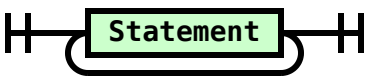
Next, we put our ideas into words. A first pass: "Programs are structured as a sequence of one or more statements, each of which is an assignment or print statement, with expressions formed with numbers, variables, function calls, and the usual arithmetic operators, which can have parentheses when needed. Comments look like the slash-slash comments of C-like languages."

Natural language isn't super precise, so as we sometimes do in design, we'll next turn thoughts into pictures.

Syntax Diagrams

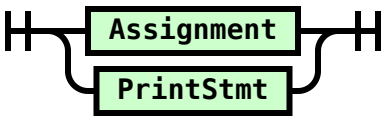
A program is a sequence of one or more statements:

Program



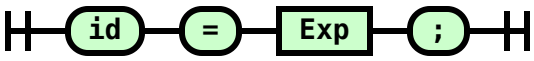
There will be two kinds of statements: assignments and print statements:

Statement

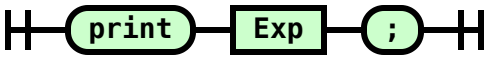


Assignments and print statements look like this:

Assignment



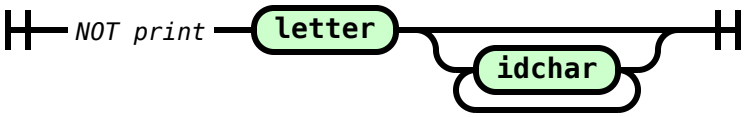
PrintStmt



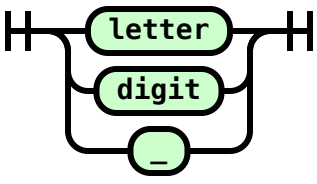
An **identifier** is the computer science word for a *name* you attach to an **entity** (a variable, constant, function, type, parameter, or similar thing). A **keyword** is a special word in your language, like `let` or `if` or `while` —these words generally do not name things—they are just used to structure the code. If a keyword is not allowed to be an identifier, we call it a **reserved word**. Astro has one reserved word: `print`.

Identifiers in Astro begin with a letter, and can have letters, digits, and underscores (examples: `x`, `last_attempt`, `p1`, `p2`, `overTheLimit`, `bot8675309_jEnNy`). We will call letters, digits, and underscores **identifier characters** (`idchar`s). But notice that `print` is not allowed to be an identifier:

id



idchar



How is the reserved word `print` specified? We have to be careful. It's not just the five letters p, r, i, n, and t. If it were then the program:

```
printy;
```

would be legal! It would be the five characters spelling print followed by a legal expression, namely the identifer y. We want the word `print` to not bleed into any following characters that might be part of an expression. In other words, `print` must not be immediately followed by an identifier character:

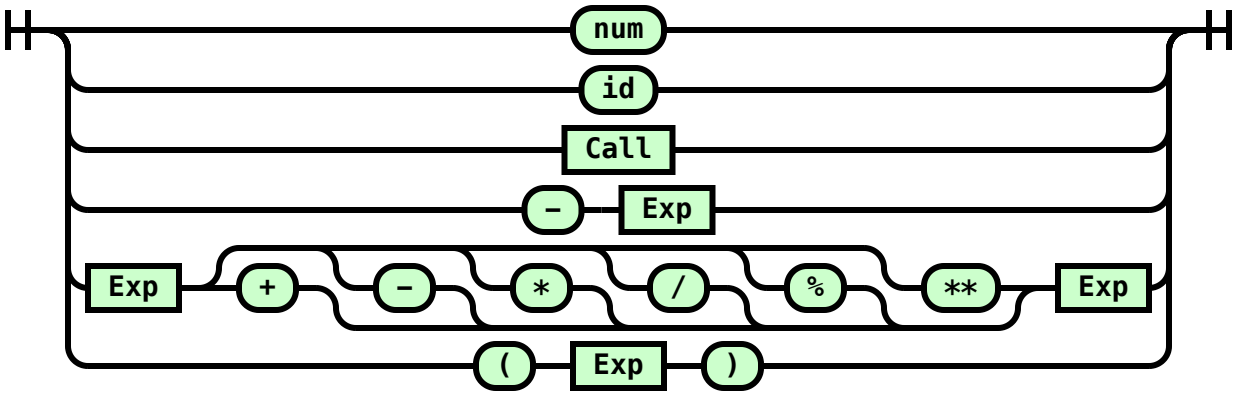
print



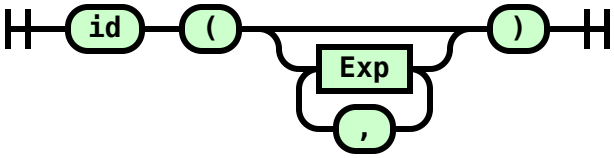
> ***Exercise**: Make sure you understand how this definition ensures that "printy" is an identifier and not* `print` *followed by the identifier* `y`.

A **statement** performs an action (perhaps with a side effect), but **expressions** represent the computation of a value. Expressions can be numbers, identifiers (representing variables), function calls, or can be made up of operators applied to other expressions. We can put parentheses around any expression. This will do the trick (but spoiler alert: we will have to change it later):

Exp



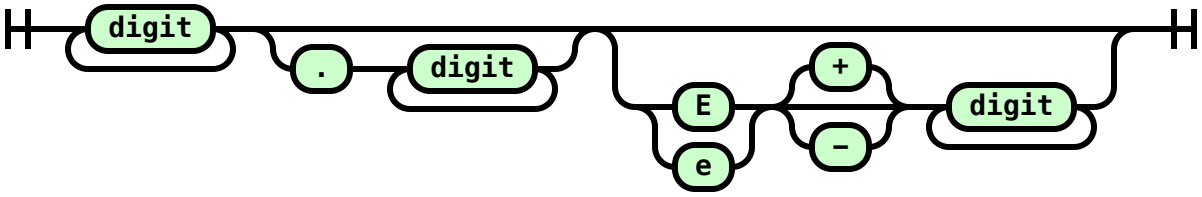
Call



> ***Exercise**: Explain the difference between expressions and statements to a friend. Make yourself a sacred vow never to mix the terms up. Precise language is a good thing. Sometimes we slip up, but still, let's try not to say things like "plus statement."*

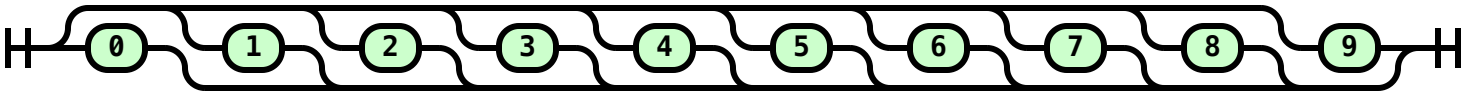
We mentioned numerals here, so let's define these. We'll keep things in decimal only (no worries about hex or binary), and use the times-ten-to-the notation from popular programming languages:

num



What about letters and digits? Well digits are pretty easy:

digit

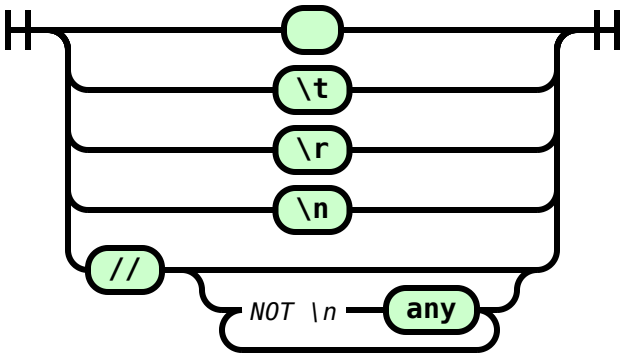


For letters, well, there are tens of thousands of these in Unicode, so we'll just say the rule for `letter` is "built-in."

Lexical vs. Phrase Syntax

Did you notice that we used two kinds of nodes in our syntax diagrams? We used ovals for constructs that internally cannot have spaces in them; we call these **lexical nodes**, or **tokens**. We used rectangles for constructs whose components can have spaces between them; we call these **phrase nodes**, or **phrases**. The special category called `space` defines what can separate tokens within phrases.

space



In the little language Astro that we are designing, the separators are the space character, the tab character, the carriage return character, the linefeed character (the linefeed (U+000A) is pretty much universally accepted as the “newline”), and comments. Comments function as spaces, too! For comments, we chose to start with `//` and go to the end of the current line.

We also followed an accepted convention to start the names of lexical nodes with lowercase letters and capitalize the names of phrase nodes.

Another convention we will follow for programming language syntax is to take the following as predefined:

- `letter`, for any Unicode letter
- `digit`, for `"0".."9"`
- `alnum`, for `letter | digit`
- `upper`, for any Unicode uppercase letter
- `lower`, for any Unicode lowercase letter
- `hexDigit`, for `digit | "a".."f" | "A".."F"`
- `any`, for any Unicode character at all

and also allow these conveniences:

- `\u{` and `}`, delimit a code point in hex and stands for the unicode character with that code point
- `\n`, for the LF character (same as `\u{a}`)
- `\r`, for the CR character (same as `\u{d}`)
- `\t`, for the tab character (same as `\u{9}`)
- `\"`, for the quote character (same as `\u{22}`)
- `\'`, for the apostrophe character (same as `\u{27}`)
- `\\`, for the backslash character itself (same as `\u{5c}`)

Note that `any` can be defined as `\u{0}..\u{10ffff}`.

> ***Remember***
>
> *The term* `space`*, though it looks like a lexical node, is something meta; it is treated specially.*

Why do we separate the lexical and phrase syntax? The lexical syntax, with the exception of the special rule for `space`, show us how to combine characters into words and punctuation into tokens (words) and the phrase syntax show us how to combine words and symbols into phrases (sentences), where the words and symbols can, if you wish, be surrounded by spacing.

> ***Exercise**: Meditate on how profound this is. Have you seen anything like this before? Hint: Are you reading this page in English?*

As a specific example, this program:

```
  print( // 麻🌿
420 );
```

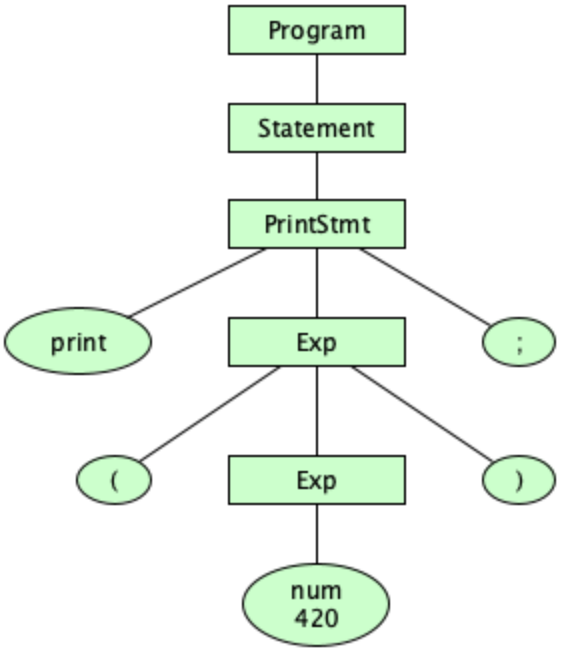
is made up of these characters:

SPACE *SPACE* *LATIN SMALL LETTER P* *LATIN SMALL LETTER R* *LATIN SMALL LETTER I* *LATIN SMALL LETTER N* *LATIN SMALL LETTER T* *LEFT PARENTHESIS* *TAB* *SOLIDUS* *SOLIDUS* *SPACE* *KANGXI RADICAL HEMP* *HERB* *LINEFEED* *DIGIT FOUR* *DIGIT TWO* *DIGIT ZERO* *SPACE* *RIGHT PARENTHESIS* *SEMICOLON*

Following the lexical syntax, skipping the spaces (and comments), we get the **token stream**:

`print` `(` `num(420)` `)` `;`

Following the phrase syntax, we can uncover the underlying **parse tree**:



A very important thing to note: **The frontier of the parse tree is the token stream**.

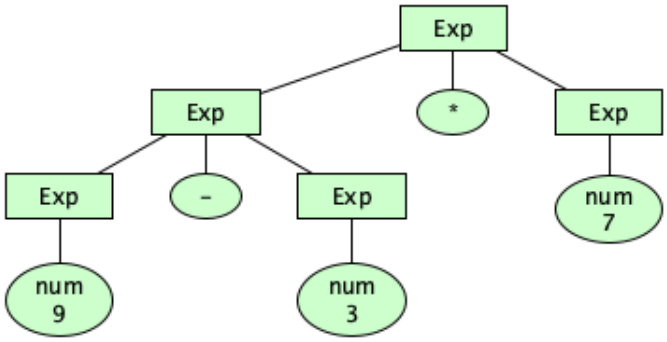
> ***Exercise**: Repeat this phrase to yourself five times: The frontier of the parse tree is the token stream.*

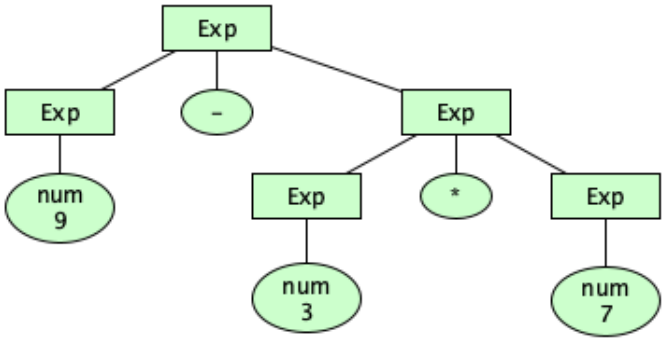
> ***The parse tree ends at tokens, not characters***
>
> *Please take the time to consider how silly it would be if the parse tree expanded all of the lexical rules down to characters. Having the parse tree stop at tokens is a good example of what we would call "breaking a complex problem down into simpler parts."*

Another term for "parse tree" is **concrete syntax tree** (CST).

Ambiguity

How are we doing so far? We are able to distinguish well-structured Astro programs from all other Unicode strings. But there are some things we haven't dealt with yet. For one, we have some strings with multiple structural forms. For example, the phrase `9-3*7` can be **parsed** in two ways:





Having more than one parse tree for a given input string means that our syntax description is **ambiguous**. It's possible to handle this particular kind of ambiguity in the syntax. Here's how.

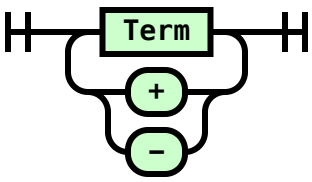
Precedence

We can create rules that force certain operators to be applied before others; that is, the **precedence** of operators can be enforced in our syntax definition. To do so, we define additional syntactic categories. We say:

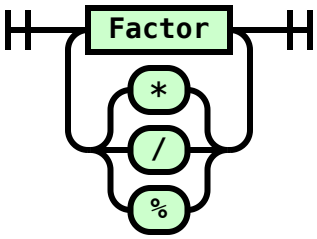
- An expression is a sequence of one or more **terms**, separated by additive operators.
- A term is a sequence of one or more **factors**, separated by multiplicative operators.
- A factor is made up of **primaries**, separated by exponentiation operators, or a factor can just be a negated primary.
- A primary is the most basic expression possible: either a simple identfier, a numeral, a call, or a parenthesized expression. Parentheses allow unlimited nesting.

We can write:

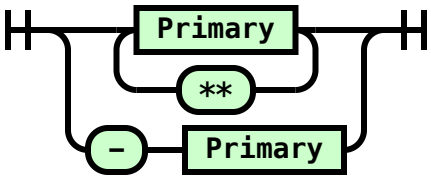
`Exp`



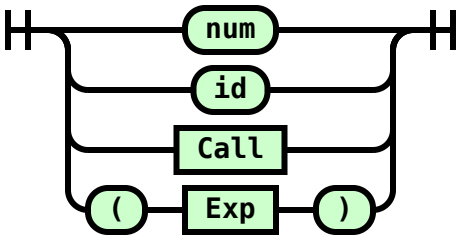
`Term`



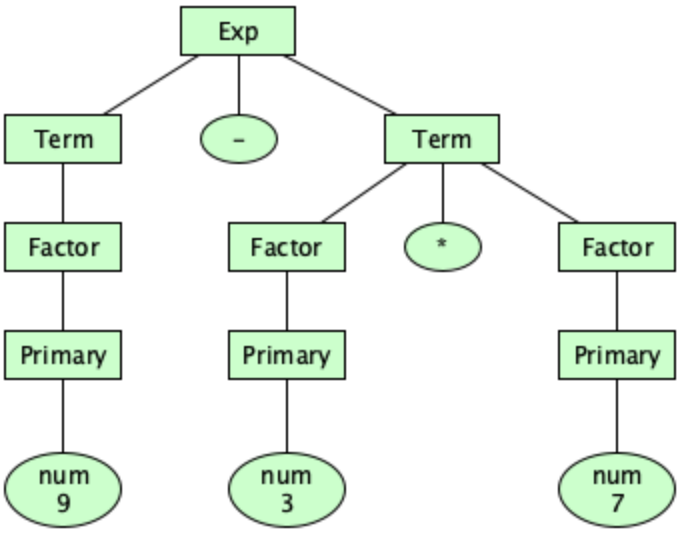
`Factor`



Primary



Great! Now there is one and only one parse tree for that previously problematic expression:



Note that the new syntax has forced the binary operators into a precedence hierarchy!

- `+` and `-` have the lowest precedence.
- `*`, `/`, and `/` have the next higher precedence.
- `**` has the highest precedence.

Of course, you can think of parenthesized expressions as being done before anything else, though we don't usually think of these as operators.

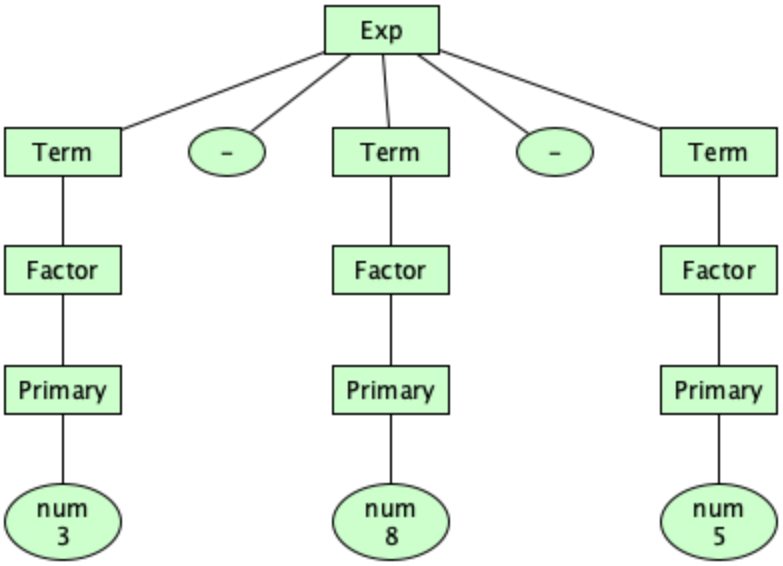
> ***Exercise**: Precedence is usually a concept that applies only to binary operators, but they way in which unary operators mix with binary operators does need to be addressed. For instance, how should* `-3**2` *be parsed? In Python exponentiation precedes negation, like* `-(3**2)`*. In Elm, negation precedes*

exponentiation, like `(-3)**2`. *Astro follows JavaScript and simply does not allow this expression (forcing programmers to use parentheses in this case)! Show how this is done.*

***Exercise**: Our syntax diagram above does not allow* `-3**2` *as an expression, but it does allow* `-3+2` *and* `-3*2`. *Why did we care only enough to ensure negation did not mix with exponentiation, but we were fine with it mixing with addition and multiplication?*

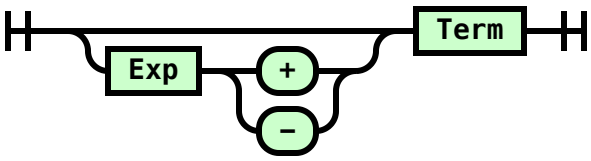
Associativity

Wait, we are not done with structuring our operators just yet. The way things stand now, the parse tree for `3-8-5` looks pretty flat:

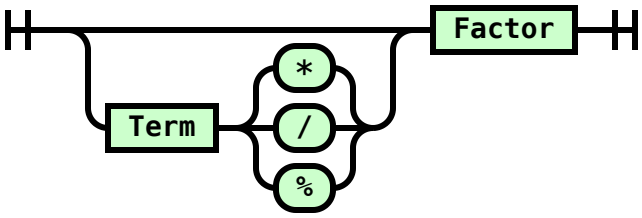


It doesn't suggest whether we mean to compute `(3-8)-5` (which would be -10) or `3-(8-5)` (which would be 0). We can give a syntax that makes this clear. In our design, let's make the additive and multiplicative operators **left-associative** and the exponentiation operator **right-associative**:

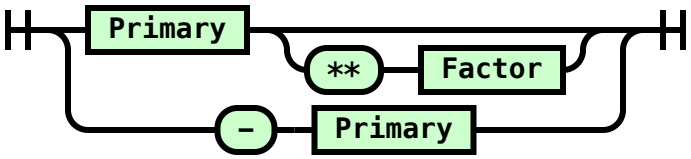
`Exp`



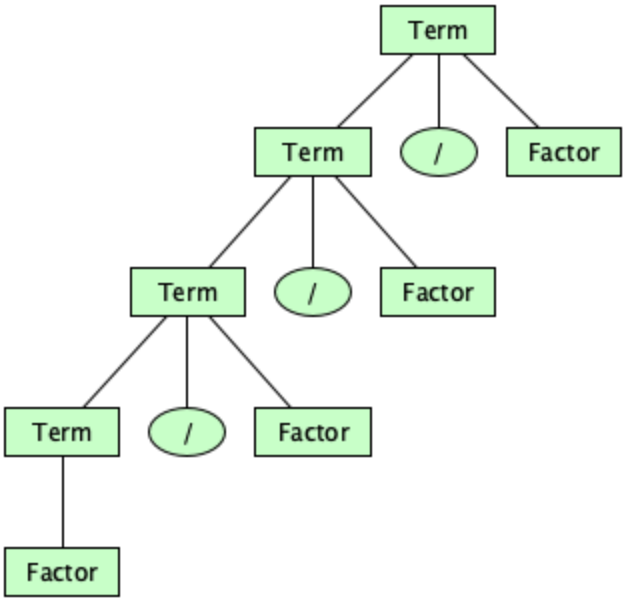
`Term`

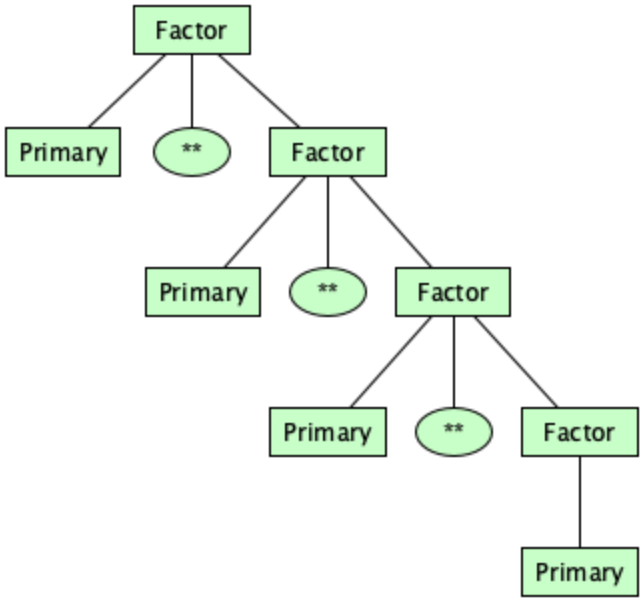


Factor



How the heck does this work? Study these parse trees, and hopefully the insight will come to you! (Hint: remember the syntax is *designed* to force the tree to "come out" a certain way.





> ***Exercise**: Study this until you understand it well.*

Grammars

Syntax Diagrams are fine for giving an overview of program structure, but, like all pictorial notations, they can take up a lot of space and are rather hard to process by a machine. If we could write a denser, text-based description of a syntax, we could write a program that checked a candidate program for conformance with the syntax! This is exactly what compilers, interpreters, and related tools do.

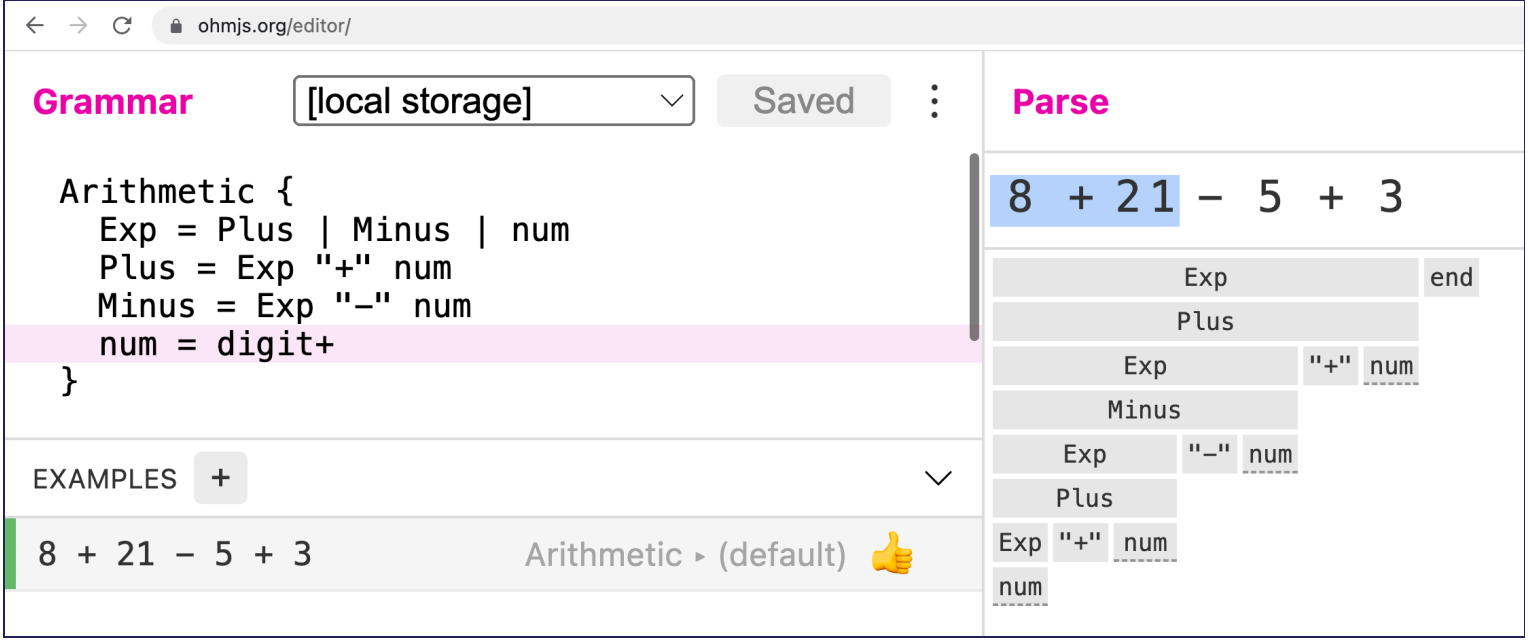
Let's translate the syntax diagrams above into a grammar for Astro. There are many different kinds of grammars that have been created for describing programming languages. We'll use an Ohm grammar for our syntax definition of Astro. We'll cover Ohm in detail later in the course, for now, we just present the grammar. Take time in your self-study to see how it matches up exactly with the syntax diagrams we've shown above.

astro.ohm

```
Astro {
  Program     = Statement+
  Statement   = id "=" Exp ";"                          --assignment
              | print Exp ";"                           --print
  Exp         = Exp ("+" | "-") Term                    --binary
              | Term
  Term        = Term ("*" | "/" | "%") Factor           --binary
              | Factor
  Factor      = Primary "**" Factor                     --binary
              | "-" Primary                             --negation
              | Primary
  Primary     = id "(" ListOf<Exp, ","> ")"             --call
              | numeral                                 --num
              | id                                      --id
              | "(" Exp ")"                             --parens

  numeral     = digit+ ("." digit+)? (("E" | "e") ("+" | "-")? digit+)?
  print       = "print" ~idchar
  idchar      = letter | digit | "_"
  id          = ~print letter idchar*
  space      += "//" (~"\n" any)*                       --comment
}
```

Keep in mind, though, that you should not just write a bunch of test programs, then write your whole grammar in one short. As design is iterative, you should *build up* the grammar from increasingly more complex examples, and test the grammar as you go. Tools will help you here! If you are using Ohm, and you should, take advantage of the use the Ohm Editor. This is an amazing tool for experimenting with programming languages.



In the upper left panel, design your grammar. You can load/save from your browser's local storage, and even publish gists to GitHub. In the lower left panel, enter test cases: both tests you want to succeed (thumbs up) and those you want to fail (thumbs down). The right panel is an interactive concrete syntax tree for the currently selected test case.

This tool will save you a lot of time.

It is an essential component of your language design toolbox.

> ***How essential is it?***
>
> *Unless your language is trivial, tools like the Ohm Editor are very important! Design is an iterative process, and creativity is enabled and enhanced with immediate feedback. So you should design with tools that allow you to experiment and test your ideas.*
>
> *That said, it is true that in practice, many production-level compilers do not use Ohm or related tools like ANTLR, Bison, etc.—they do everything by hand. But developers that go this route will write their grammar tests concurrently with their design.*

CLASSWORK

Let's do a code-along with the Ohm Editor for developing the Astro grammar. During the code-along, note how examples are done first, and note how we will evolve from the basics to more complex features, bringing in notions such as a precedence and associativity where needed.

During the code-along, bits of Ohm will be introduced as needed. In a subsequence course unit, we will cover Ohm in detail.

Defining the Statics

Question: Does the grammar we defined above for Astro capture the following (desirable) rule:

> "You can only use an identifier if it has been previously assigned to."

Answer: It does not.

***Exercise**: Show that it does not. Hint: Is* `print x;` *a legal program according to the grammar?*

***Exercise**: Show how to capture the rule in a grammar. Hint: Do not be frustrated if you cannot.*

Enforcing this rule requires knowledge of **context**. That is, the syntactic rule for producing expressions such as calls and arithmetic operations would need to somehow know which identifiers appeared on the left hand side of some previous assignment statement. This turns out to be so hard that even designers of real programming languages omit enforcement of contextual rules from the grammar! In fact, while the official grammar for Java will not derive this program:

```
class A {2 / == {{;
```

and report a **syntax error**, a Java compiler will say that this program:

```
class A {int x = y;}
```

is structurally well formed according to the official syntax of the Java language! The compilation unit consists of a type declaration that is a class declaration whose body consists of a field declaration with a type, a name, and an initializing expression which is an identifier. But we know this program is not legal, since the identifier `y` has not been declared. It's not only Java that has grammars overspecifying things: pretty much every programming language uses a grammar to define structural rules only, and specifies contextual rules in prose, or in a separate semantic definition.

For Astro, we will "define" the following contextual rules:

1. The following identifiers are built-in:

    - `π`, a number
    - `sqrt`, a function of exactly one argument
    - `sin`, a function of exactly one argument
    - `cos`, a function of exactly one argument
    - `hypot`, a function of exactly two arguments
2. An identifier cannot be used in an expression unless it is one of the built-in identifiers or has been previously assigned to.
3. All function calls must accept the proper number of arguments.
4. The built-in identifiers cannot be assigned to.
5. Identifiers declared as functions can only be called, not used in any other context. Identifiers declared as numbers cannot be called.

For more complex languages, the statics definition (contextual rules) can be quite large. Here are some things that might appear:

- Identifiers have to be declared before they are used.
- Identifiers may not be used in a place where they may have not yet been initialized.
- Identifier declarations must be unique within a scope, unless a language provides overloading.
- All expressions must be used according to their type.
- Identifiers must be used according to (1) their **scope**, (2) their **access modifiers** (private, protected, package, public, whitelist, blacklist, ...), and (3) any other meta-level attributes (enumerable, configurable, writable, deletable, callable, ...).
- Arguments must match up with parameters in terms of number, order, name, mode, and type.
- `break` and `continue` statements may only appear in a loop. `return` statements may only appear in a function it's *possible* to encode these restrictions in the grammar, but it would be ugly).
- All paths through a function must have a return.
- All possible matches in a pattern match expression, or a switch statement, must be covered.
- When inheriting from a base class with abstract methods, or implementing an interface, all abstract methods must be implemented, or the derived class must be declared abstract.
- All declared local variables must be subsequently read, and declared private functions must be called.

Defining the Dynamics

The dynamics for most programming languages are given in prose. If you language is simple enough, a formal semantic definition is possible. For the sample languages in this course, Astro and Bella are

given both informal and formal semantic definitions. We'll not be studying formal semantics at this time, but feel free to study the definitions on your own.

Prototyping

During your language design, you will want to whip up a simple interpreter to at the very least make sure your design is reasonable. You may wish to developer your interpreter in parallel with your language design.

Ohm was designed for prototyping programming languages, so it is a natural choice. The Ohm Editor, as we just saw, helps you design the syntax. To write an actual interpreter, we'll have to go much deeper into Ohm. We'll be doing this in our next unit of study in this course, which is, indeed, a deep study of Ohm.

Examples

Many well-known programming languages have published, formal definitions. You can find them by searching the web.

For this class, we will be studying five little languages crafted especially to help you in your stufy of language design and implementation. We will be studying them in order, building upon previous languages and learning new things as we progress. This will allow us to introduce the huge topic of language processing in a practical setting, writing real compilers for real languages. The languages are Astro, Bella, Carlos, Dax, and Ekko.

Astro



We all begin as a white belt in every new endeavor. We will start, then, with a very simple, almost trivial, language. All it has are numbers, arithmetic operators, variables, and a few pre-defined constants and functions. Here's an example program:

```
// A simple program in Astro
rAd1uS = 55.2 * (-cos(2.8E-20) + 89) % 21;
the_area = π * rAd1uS ** 2;
print(hypot(2.28, 3 - rAd1uS) / the_area);    // woohoo 👻
```

There are only two kinds of statements: assignments and print statements. Expressions include numbers, variables, function calls, arithmetic expressions with `+`, `-`, `*`, `/`, `%`, and `**`, and can be parenthesized. We will cover the official definition of the language, and use the language to motivate a formal study of syntax.

When studying this language, we'll learn about the separation of context-free syntax from contextual rules. Contextual rules include such things as: having to match the number of arguments in a call with the number of defined parameters, rudimentary type checking, and not allowing assignments to read-only variables.

As Astro will be our first language, we will use it as a case study to learn the amazing Ohm Language Library to build an interpreter. The details of how the interpreter is constructed are covered in the course notes on Ohm.

Bella

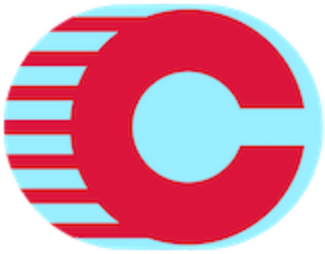


Our second language has a few things Astro does not: a richer set of operators, variable declarations, and user-defined functions. The contextual rules for Bella are much richer than that of Astro, since we now have actual declarations, and scope! Here's an example program:

```
let dozen = 12;
print dozen % 3 ** 1;
function gcd(x, y) = y == 0 ? x : gcd(y, x % y);
while dozen >= 3 || (gcd(1, 10) != 5) {
  dozen = dozen - 2.75E+19 ** 1 ** 3;
}
```

We will first look at the official specification, introducing all sorts of interesting concepts. Then we'll study a real, actual Bella compiler. Here we learn about designing and architecting a compiler, building the components (analyzer, optimizer, and generator), and getting 100% test coverage. The compiler source code is on GitHub.

Carlos



In our third language, we encounter arrays, structs, and optionals: our first language that is basically useful. If you are taking the compiler course for which these notes were written, Carlos is a good example of the minimal language complexity you will need for your term project.

```
const languageName = "Carlos";

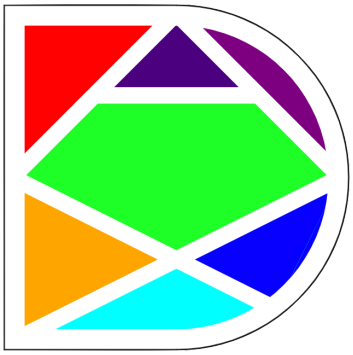
function greeting() {
  return random(["Welcome", "こんにちは", "Bienvenido"]);
}

print("👋👋👋");
repeat 5 {
  print(greeting() + " " + languageName);
}
```

We'll be visiting the language's official specification and a compiler on GitHub. The compiler (of course!) uses the Ohm Language Library.

There's no separate page of notes describing the compiler. After studying the Astro and Bella compilers, you'll be able to find your way around the code on GitHub (there's documentation). And don't worry, it's development and usage will be covered in class, and the teaching staff can help you with any questions you might have.

Dax



Language number four is a functional language, that is, a language with no assignments! The only bindings of names to entities happens when passing arguments to parameters, though there is that famous `let` declaration which nicely sugars a function call: it's nicer to say `let x = 5 in x * y end` than `{x => x * y}(5)`.

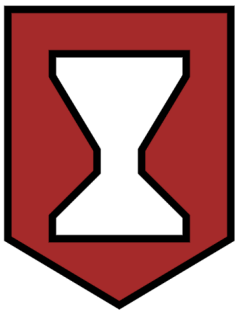
Here's a sample program to get the feel for the language:

```
let
  gcd = {x => {y ==> y == 0 ? x : gcd y (x % y)}};
  z = 5
in
  [1, 3, z] |> filter {x => x > 2} |> map {x => x ** 2} |> print
  then
  "hello" |> substring 2 5 |> print
  then
  print (gcd 33 99)   // This is fine, you don't HAVE to use |>
end
```

If you have not yet seen languages with the awesome `|>` operator, here's your chance to be wowed.

We will discuss the language design and compiler later in the course.

Ekko



Our fifth language, Ekko (starting with E like Erlang and Elixir, which greatly influence it), is a kind of experimental language that deals quite a lot with **time**.

Ekko mixes styles of asynchronous programming from JavaScript and the distributed process-orientation of Erlang and Elixir: Ekko's `future` objects are based on JS promises, and its processes communicate via messages as in Erlang. There's also quite a bit more temporal goodness, including timeout calls, value histories with time travel (influenced by older versions of Elm), and even explicit parallelism.

More Examples

Students in previous iterations of the course have designed and implemented their own languages. Here's a sampling of language over the past decade or so. (Please note that there is a very wide variety of quality in these examples. They are presented here without any evaluative commentary as to whether they are suitable building blocks for one's own project.)

- Blvd
- Sous
- River
- Rat
- Cod
- GLHF
- BuildLang
- Banana

- Pyth-on-Point
- Dino
- Sanscript
- Kis
- YALL*E
- Jazz
- Barista
- LangY
- MODE
- Bang
- Squishi
- SussyScript
- CoffeeMaker
- YeeHaw
- Gullienne
- ElectricScript
- Barista
- Shrek--
- Chinese Baozi
- IcedCoffee
- Chloe
- T.O.A.L.
- Pirates Code
- Leg
- Kobe
- Cringe--
- Bismath
- Plumb
- PokerScript
- Pandemonium
- Easi
- Palet
- VikingScript
- Speare
- Ahtohallan
- VineScript
- Big Mama's Kitchen
- Medley
- Custom
- LemonScript
- Aegis
- Midi-Cholorian
- qcq
- HYPER!
- RealHotGirlScript
- Pivot
- SnekQL
- Respec++
- r0b0p
- uwuScript
- PawvaScript
- Cuttlefish
- Inkling
- Casper
- StoneScript
- FUZZ++
- Goof3
- Scriptofino
- 101Script
- Jen
- Olive
- Nebula
- J4
- Alula
- Pycante
- Whiteboard
- Favascript
- Queen
- MemeScript
- Skrt
- JCaml
- Boozie
- Deeg
- AvaJava
- Yah
- Roo
- Agate
- AboveAverageScript
- TeaScript
- YACBL
- Yoda
- Spitfire
- Derpodile
- GollumScript
- Singularity
- KobraScript
- Min

Recall Practice

Here are some questions useful for your spaced repetition learning. Many of the answers are not found on this page. Some will have popped up in lecture. Others will require you to do your own research.

What are the four phases of programming language design? HIDE ANSWER

(1) Working out the desired context (audience, purpose, scope), (2) sketching example programs, (3) formalizing the syntax and semantics, and (4) prototyping.

Name five programming language paradigms. HIDE ANSWER

A few are: imperative, declarative, structured, object-oriented, functional, applicative, concatenative, logic, protocol-oriented, aspect-oriented, array, event-driven, dataflow, agent-based. There are others

What were the four big ideas in programming languages mentioned in Bret Victor's *Future of Programming* talk? HIDE ANSWER

(1) Direct manipulation of data rather than showing the code, (2) Goals and constraints rather than procedures, (3) Spatial representation of code rather than textual representations, and (4) Concurrent computation rather than sequential

What language did Alan Kay feature in his Programming Languages video? HIDE ANSWER

eToys

What are some things to decide upon while undertaking language design? HIDE ANSWER

The audience, the purpose, the scope, the problem domain it is target to, whether it is reasonable or esoteric, whether it has a simple foundation or favors pragmatism, its concurrency model.

In what essay did Bret Victor lay out several principles that languages and programming enviroments should follow to aid their users' learning? HIDE ANSWER

Learnable Programming

What is abstract syntax? HIDE ANSWER

The structure of a program without regard to the specific syntax used to represent it.

What is the name of the specification of JavaScript's abstract syntax? HIDE ANSWER

ESTree

At what web address can you find and interactive JavaScript AST builder? HIDE ANSWER

https://rtoal.github.io/js-ast/

What are five approaches of showing (concrete) syntactic structure in popular programming languages? HIDE ANSWER

Curly braces, indentation, terminal-ends, parentheses, postfix

What is an example of a programming language for which newlines are significant? HIDE ANSWER

Answers include Python and Ruby

What is syntactic sugar? HIDE ANSWER

Syntax within a programming language that is designed to make certain phrases more clear, concise, or elegant than the basic forms defined in the language.

What is syntactic salt? HIDE ANSWER

Syntax within a programming language that is designed to make certain phrases more confusing, verbose, or awkward than the basic forms defined in the language.

What is a candygrammar? HIDE ANSWER

A syntax that looks sweet and natural language-like that appears good but turns out to be bad for you.

What is a golfing language? HIDE ANSWER

A language designed to be as terse as possible, often for the purpose of helping you win shortest-possible code challenges.

What are some examples of golfing languages? HIDE ANSWER

CJam, Pyth, Stax, GolfScript, 05Ab1E

What are three styles of programming language definition? HIDE ANSWER

Informal, formal, and executable

What is the difference between syntax and semantics? HIDE ANSWER

Syntax deals with program structure. Semantics deals with program meaning.

What is the difference between a language's **statics** and its **dynamics**? HIDE ANSWER

Statics refers to the rules that can be checked at compile time. Dynamics refers to the rules that can only be checked at run-time.

What are syntax definitions generally split into lexical and phrase syntaxes? HIDE ANSWER

Lexical syntax defines how invidividual characters are grouped in to words (tokens), while phrase syntax defines how these tokens are combined to form larger structures.

What is a parse tree? HIDE ANSWER

A tree that describes the syntactic structure of a program as defined by the language's syntax.

The frontier of the parse tree is the program's _____. HIDE ANSWER

Token stream

A parse tree is also called a _____. HIDE ANSWER

Concrete syntax tree

What does it mean for a syntactic specificiation to be ambiguous? HIDE ANSWER

There exists at least on program that has more than one parse tree.

What attributes of operators are generally used to disambiguate a syntactic specification? HIDE ANSWER

Precedence and associativity

Because syntax diagrams may take up a lot of space, what alternaive method of syntactic specification is typically used? HIDE ANSWER

Grammars

What tool is provided by the authors of the Ohm library to help you prototype a new language design? HIDE ANSWER

The Ohm Editor

What was the title of Alex Warth's dissertation? HIDE ANSWER

Experimenting with Programming Languages

What are some examples of legality rules that are considered too hard to be captured in a grammar and are thus typically defined within a program's static semantics? HIDE ANSWER

You can only use an identifier if it has been previously assigned to, all function calls must accept the proper number of arguments, the built-in identifiers cannot be assigned to, identifiers declared as functions can only be called, not used in any other context, identifiers declared as numbers cannot be called, type checking, access checking (private, public), declared identifiers must be initialized or must be read. (There are many more.)

What example programming languages have been designed for this course? HIDE ANSWER

Astro, Bella, Carlos, Dax, Ekko

Summary

We've covered:

- ✅ The cycle of language design phases
- ✅ What to know before undertaking language design
- ✅ Pointers to excellent articles and essays about language design
- ✅ Two videos (one by Alan Kay, one by Bret Victor) on languages and language design
- ✅ How to begin the language design process
- ✅ Questions to ask while designing your language
- ✅ Concepts and features to think about when sketching during design
- ✅ Abstract Syntax
- ✅ Concrete Syntax
- ✅ The use of the Ohm Editor in language prototyping
- ✅ (Formal) Language Definition
- ✅ Brief notes on five example languages

Ohm



Language definitions should not be written on paper alone. They should be brought to life. There are many libraries out there that will help you define and process (parse, analyze, generate) languages. Ohm is but one of them, but it's totally worth studying.

CONTENTS

What is Ohm? • Getting Started • Ohm Basics • Official Documentation • Example Grammars • Example Applications • More Examples • Recall Practice • Summary

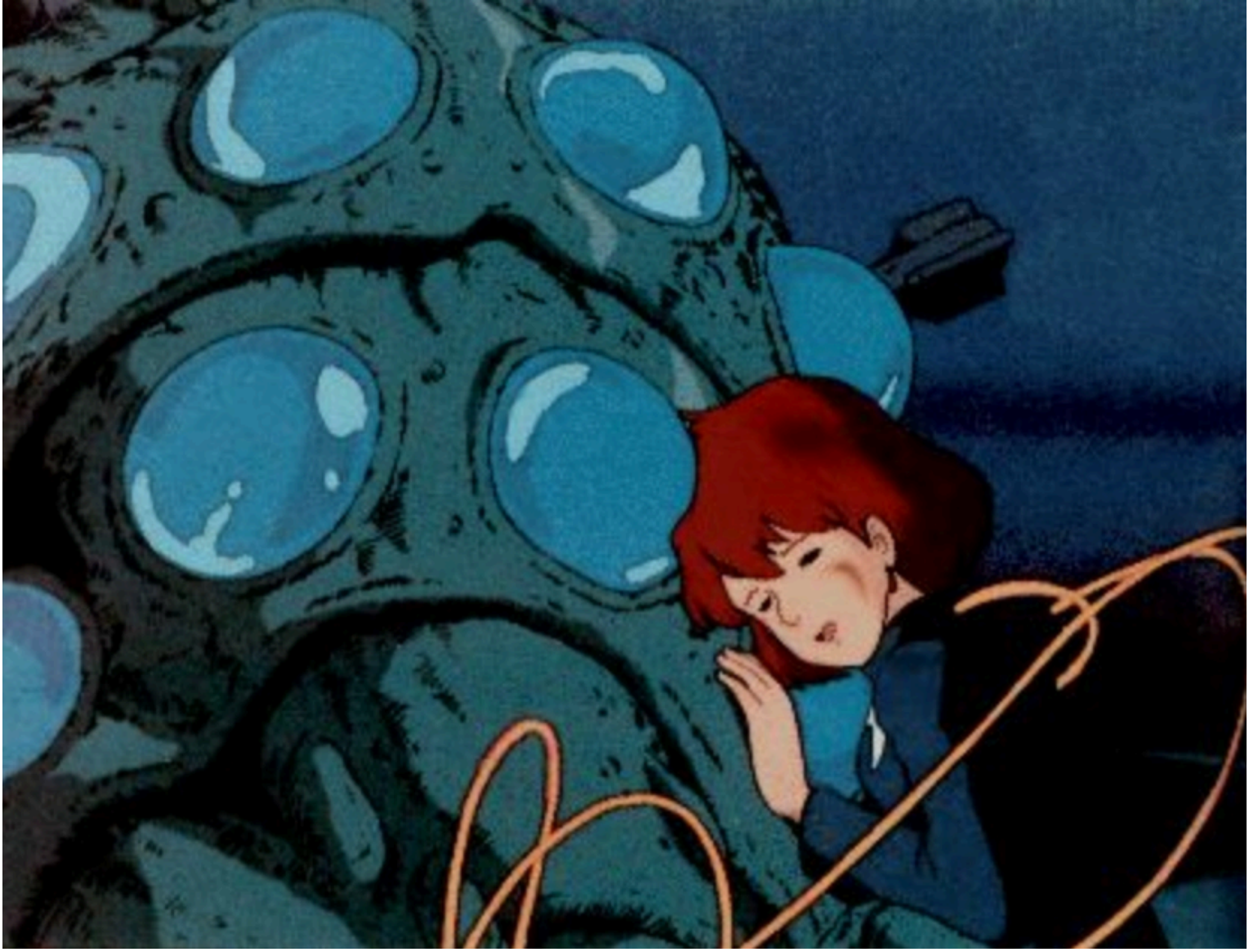
What is Ohm?

Ohm is advertised as a library for building parsers, interpreters, compilers, and more. With it you can quickly generate parsers simply by writing a grammar, then write whatever actions you like to work with the parser output. So with just one grammar you can write:

- A compiler (with multiple targets, even!)
- An interpreter
- A semantics definition
- A syntax highlighter

Yes, you can use Ohm to provide both a formal syntax of a language and a formal semantics (in the form of an interpreter!). Both the syntax definition and the semantics definition are executable in a real sense. Not just math equations on paper!

There's even an interactive tool that enables to to interactively design grammars so that you can prototype new and beautiful programming languages!



Ohm may not generate the fastest parsers but its cleanliness, simplicity, and flexibility make it stand out from the competition.

> ***Ohm Home Page***
>
> *You can find all about Ohm, obtain the source code, read all of its documentation, and find links to tutorials, examples, real-world projects, and the community Discord server, at the home page.*

> ***Ohm Tutorial***
>
> *Pat Dubroy, a co-creator of Ohm, wrote a nice tutorial. It's worth a look as well.*

Getting Started

Here's a short video in which we:

1. Introduce the Ohm Editor and design a grammar for a tiny programming language of arithmetic expressions and print statements
2. Write an interpreter for this language so that one can run programs directly on the command line.

By the way: if you can write a complete interpreter for a language, you can claim to have written a semantics—if you accept that it's fine to say an interpreter *is* the semantics!



Here's the result of the code-along from the video. If you did not watch the video, or don't care to watch it, skip ahead to the Ohm Basics section.

```
import * as ohm from "ohm-js";
import fs from "fs";

const grammar = ohm.grammar(String.raw`PlusMinusLanguage {
  Program   = Statement+
  Statement = "print" "(" Exp ")" ";"
  Exp       = Exp "+" Term                  --plus
            | Exp "-" Term                  --minus
            | Term
  Term      = "(" Exp ")"                   --parens
```

```
            | num
  num       = digit+
}`);

const interpreter = grammar.createSemantics().addOperation("eval", {
  Program(statements) {
    for (const statement of statements.children) {
      statement.eval();
    }
  },
  Statement(_print, _leftParen, expression, _rightParen, _semicolon) {
    console.log(expression.eval());
  },
  Exp_plus(left, _plus, right) {
    return left.eval() + right.eval();
  },
  Exp_minus(left, _plus, right) {
    return left.eval() - right.eval();
  },
  Term_parens(_leftParen, expression, _rightParen) {
    return expression.eval();
  },
  num(digits) {
    return Number(digits.sourceString);
  },
});

const match = grammar.match(fs.readFileSync(process.argv[2]));
if (match.failed()) {
  console.error(match.message);
} else {
  interpreter(match).eval();
}
```

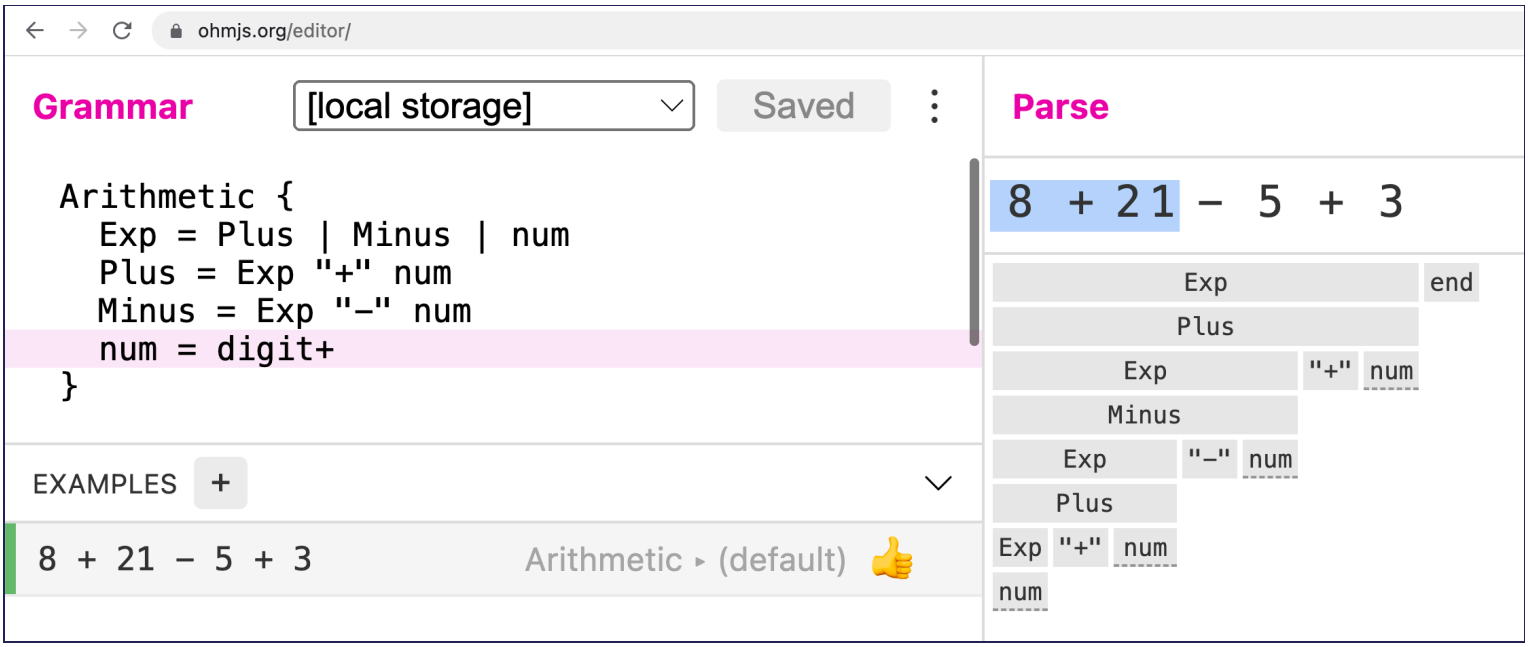
Ohm Basics

The best way to get started is to use the amazing Ohm Editor. While composing your grammar, write and execute lots of text cases. The tests will light up green for success and red for failure as you type. The parse tree will also appear as you type. The immediate feedback cycle is the best way to create.

Watch the video above for details, or if you prefer to save yourself the watching time and just want to read, try out the following famous grammar for starters:

```
Arithmetic {
  Exp   = Plus | Minus | num
  Plus  = Exp "+" num
  Minus = Exp "-" num
  num   = digit+
}
```

Write some test cases and explore the parse trees. Note how the operators are left associative.



Now, let's go through the basics in more detail, and build an actual interpreter!

Setting up a Project

Let's make an interpreter for a real programming language, Astro. Begin as you would to make any Node.js project (with a recent version of Node.js, of course):

```
$ mkdir astro-interpreter
$ cd astro-interpreter
```

```
$ npm init -y
```

Then edit package.json to have a top-level property `"type": "module"`.

Include Ohm in your project:

```
$ npm install ohm-js
```

Check package.json to make sure the Ohm package is there now.

Writing a Grammar

Create the file **astro.ohm** and populate it with the grammar:

astro.ohm

```
Astro {
  Program     = Statement+
  Statement   = id "=" Exp ";"                          --assignment
              | print Exp ";"                           --print
  Exp         = Exp ("+" | "-") Term                    --binary
              | Term
  Term        = Term ("*" | "/" | "%") Factor           --binary
              | Factor
  Factor      = Primary "**" Factor                     --binary
              | "-" Primary                             --negation
              | Primary
  Primary     = id "(" ListOf<Exp, ","> ")"             --call
              | numeral                                 --num
              | id                                      --id
              | "(" Exp ")"                             --parens

  numeral     = digit+ ("." digit+)? (("E" | "e") ("+" | "-")? digit+)?
  print       = "print" ~idchar
  idchar      = letter | digit | "_"
  id          = ~print letter idchar*
  space      += "//" (~"\n" any)*                       --comment
}
```

Design with the Ohm Editor

Especially when working with larger languages, the process of interactively designing with a live tool like the Ohm Editor is pretty much a necessity. Once designed, you'll want to grab your grammar from the editor window and put it into a real software project.

Understanding Parsing Expressions

Ohm grammars consist of a name and a number of parsing expressions. Details are in the syntax reference, which you need to eventually read. But as you may be impatient, here are the Ohm parsing expressions:

Expression	Description
`" ... "`	A literal string. Allows the usual escape chars `\'`, `\"`, `\b`, `\n`, `\r`, `\t`, `\f`, `\\`, `\x` ***hh***, `\u` ***hhhh***, and `\u{` ***h...h*** `}`.
rulename	Just a rule.
rulename < ***params*** >	A rule with parameters.
e *	***e***, zero or more times.
e +	***e***, one or more times.
e ?	***e***, zero or one times.
e_1 e_2	Match e_1 then match e_2.
e_1 | e_2	If e_1 matches, great. Otherwise try to match e_2.
& ***e***	Match ***e*** without consuming it (lookahead).
~ ***e***	Succeed if ***e*** doesn't match, without consuming anything (negative lookahead).
# ***e***	Lexify a syntactic rule.

Understanding Lexical and Syntax Rules

Lexical rules always start with lower case letters and **syntax rules** start with uppercase letters. Syntax rules allow tokens from the class `space` to be implicitly interspersed with other tokens.

Built-In Rules

There are several built-in rules:

- `any` = *any char at all*
- `end` = *the end of the input*
- `space` = `" " | "\x09".."\x0d" | "\ufeff" |` *any char from Unicode category Zs*
- `digit` = `"0".."9"`
- `lower` = *any char from Unicode category Ll*
- `upper` = *any char from Unicode category Lu*
- `unicodeLtmo` = *any char from Unicode category Lt, Lm, or Lo*
- `letter = lower | upper | unicodeLtmo`
- `alnum = letter | digit`
- `hexDigit = digit | "A".."F" | "a".."f"`
- `caseInsensitive<t>` = *any case insensitive version of t*
- `emptyListOf<elem, sep> = /* nothing */`
- `nonemptyListOf<elem, sep> = elem (sep elem)*`
- `listOf<elem, sep> = nonemptyListOf<elem, sep> | emptyListOf<elem, sep>`
- `EmptyListOf<elem, sep> = /* nothing */`
- `NonemptyListOf<elem, sep> = elem (sep elem)*`
- `ListOf<elem, sep> = NonemptyListOf<elem, sep> | EmptyListOf<elem, sep>`

Rule Arities are Important in Ohm

All alternatives for a rule must have the same arity. This is a small price to pay for the massive advantage you get from separating the grammar from the parsing actions that we will see later. But there is a cool short-hand to make it look like alternatives with different arities exist. For example:

```
Exp = Exp "+" Term  --plus
    | Exp "-" Term  --minus
    | Term
```

is an abbreviation for:

```
Exp       = Exp_plus | Exp_minus | Term
Exp_plus  = Exp "+" Term
Exp_minus = Exp "-" Term
```

Grammar Inheritance

Grammars can inherit from other grammars. When inheriting, use:

- `=` to add a new rule
- `:=` to replace the rule from the supergrammar
- `+=` to add an alternative to the rule from the supergrammar

Any grammar you write yourself automatically inherits from the base Ohm grammar, where the built-in rules are defined.

Match Results

Given a grammar g and source code c, g`.match(`c`)` returns a `MatchResult` object, m. If m`.succeeded()` is true, m wraps a parse tree, and you can apply a semantics object to it. If m`.failed()` is true, a detailed error message will be in m`.message` and a smaller error message will be in m`.shortMessage`.

In the Astro interpreter we're building, create the file **astro.js**. This file will house the program. We can get started by populating it with:

```
import * as ohm from "ohm-js"
import fs from "fs"

const astroGrammar = ohm.grammar(fs.readFileSync("astro.ohm"))
const match = astroGrammar.match(process.argv[2])
if (match.failed()) throw new Error(match.message)
console.log("Syntax ok")
```

In your terminal, invoke

```
$ node astro.js 'print(3 + 5.5);'
```

And you should see the response:

```
Syntax ok
```

Applying Semantics

Once we have a parse tree we can process it with what Ohm calls a **semantics object**. You can write a semantics object to do *anything you want* with your parse tree, which is why Ohm is awesome.

> ***Ohm's Superpower***
>
> *In other language processing libraries, like ANTLR and Bison, semantic processing rules are embedded inside the grammar. Ohm clearly separates them. This makes it easy in Ohm to write many sets of semantic actions for a single grammar.*

The thing we are going to do with these notes is write an interpreter.

A semantics object is a collection of operations and attributes. Operations and attributes are applied to each node in the parse tree. Operations and attributes are similar: operations are functions (called every time), and attributes are memoized and called with property syntax. There are default things to do when a semantics object doesn't have operations or attributes for all the rules. There's a lot going on, because Ohm is so powerful. So let's just jump right in to adding an interpreter to our running example.

Here's the complete **astro.js**:

astro.js

```
import * as ohm from "ohm-js"
import fs from "fs"

const astroGrammar = ohm.grammar(fs.readFileSync("astro.ohm"))

const memory = {
  π: { type: "NUM", value: Math.PI, mutable: false },
```

```
  sin: { type: "FUNC", value: Math.sin, paramCount: 1 },
  cos: { type: "FUNC", value: Math.cos, paramCount: 1 },
  sqrt: { type: "FUNC", value: Math.sqrt, paramCount: 1 },
  hypot: { type: "FUNC", value: Math.hypot, paramCount: 2 },
}

function check(condition, message, at) {
  if (!condition) throw new Error(`${at.source.getLineAndColumnMessage()}${messag
}

const evaluator = astroGrammar.createSemantics().addOperation("eval", {
  Program(statements) {
    for (const statement of statements.children) statement.eval()
  },
  Statement_assignment(id, _eq, expression, _semicolon) {
    const entity = memory[id.sourceString]
    check(!entity || entity?.type === "NUM", "Cannot assign", id)
    check(!entity || entity?.mutable, `${id.sourceString} not writable`, id)
    memory[id.sourceString] = { type: "NUM", value: expression.eval(), mutable: t
  },
  Statement_print(_print, expression, _semicolon) {
    console.log(expression.eval())
  },
  Exp_binary(left, op, right) {
    const [x, y] = [left.eval(), right.eval()]
    return op.sourceString == "+" ? x + y : x - y
  },
  Term_binary(left, op, right) {
    const [x, o, y] = [left.eval(), op.sourceString, right.eval()]
    return o == "*" ? x * y : o == "/" ? x / y : x % y
  },
  Factor_binary(left, _op, right) {
    return left.eval() ** right.eval()
  },
  Primary_parens(_leftParen, e, _rightParen) {
    return e.eval()
  },
  Primary_num(num) {
    return Number(num.sourceString)
```

```
  },
  Primary_id(id) {
    const entity = memory[id.sourceString]
    check(entity !== undefined, `${id.sourceString} not defined`, id)
    check(entity?.type === "NUM", `Expected type number`, id)
    return entity.value
  },
  Primary_call(id, _open, exps, _close) {
    const entity = memory[id.sourceString]
    check(entity !== undefined, `${id.sourceString} not defined`, id)
    check(entity?.type === "FUNC", "Function expected", id)
    const args = exps.asIteration().children.map(e => e.eval())
    check(args.length === entity?.paramCount, "Wrong number of arguments", exps)
    return entity.value(...args)
  },
})

try {
  const match = astroGrammar.match(process.argv[2])
  if (match.failed()) throw new Error(match.message)
  evaluator(match).eval()
} catch (e) {
  console.error(`${e}`)
}
```

CLASSWORK

> Lots to explain here!

Running the Interpreter

Let's see the program in action:

```
$ node astro.js 'print(3 + 5.5);'
8.5

$ node astro.js 'x = 5;
y = 3;
print(x ** y * sqrt(100));'
1250
```

```
$ node astro.js '
dozen = 5 + 8 - 1;
print(dozen ** 3 / sqrt(100));
print(sqrt(5.9 + hypot(π, 3.5e-8)));
'
172.8
3.0069241183624493

$ node astro.js '
print(5);
print(x - 10);
y = 100 + x;'
5
Error: Line 3, col 8:
  2 |  print(5);
> 3 |  print(x - 10);
             ^
  4 |  y = 100 + x;
x not defined
```

Note the interpreter accepts the program to interpret as a command line argument. If your Astro program were in a file (say, hello.astro) you can still run the interpreter with:

```
$ cat hello.astro | xargs -0 node astro.js
```

***Exercise**: Change the interpreter so that it interprets a program stored in a file whose name is given as a command line argument.*

Understanding Action Dictionaries

To a grammar, you attach a semantics object. To a semantics object, you can attach operations and attributes. The main property of an operation is an object called an **action dictionary**. The methods of the action dictionary correspond to rules of your grammar, with the left hand side as the method name and the right hand side as the method parameters, which are **parse nodes**. We saw this above.

But we need practice, and more examples. Here's a sample grammar that illustrates the important things to know:

```
Sample {
  S = A B* C+ D? ListOf<E, ","> f
  ...
}
```

Let's set up a semantics object with an operation called `rep`:

```
const semantics = grammar.createSemantics().addOperation("rep", { ... })
```

We'll need a method for `S`. It will have six parameters, each of which are parse nodes. There are three types of parse nodes: (1) **rule application nodes** (for phrase rules), (2) **terminal nodes** (for lexical rules and literal strings), and (3) **iteration nodes** (when `*`, `+`, and `?` are used):

```
S(a, bs, cs, ds, es, f) {
  // a is a *rule application* node
  // bs is an *iteration node* for the (0 or more) bs
  // cs is an *iteration node* for the (1 or more) cs
  // ds is an *iteration node* for the (0 or 1) ds
  // es.asIteration() gets you the *iteration node* for the es
  // f is a *terminal* node
}
```

In the body of `S`:

- Calling `a.rep()` will invoke the method `A`.
- Calling `f.rep()` will invoke the method `f` if it exists; if it does not, it will invoke the special method `_terminal`. By default, an implementation of `_terminal` already exists: it returns `this.sourceString`, but you can override this if you want.
- Calling `bs.rep()` will invoke the method `_iter` on the children of `bs`. The method `_iter` is NOT defined by default! You *could* define it yourself like

  ```
  _iter(...children) {
    return children.map(c => c.rep());
  }
  ```

  but it is much better to write things like `myNode.children.map(c => c.rep())` directly each time you have to process an iteration node, since it makes clear to a reader that you have an iteration!

Don't forget to use `asIteration` on your `ListOf` (and similar) nodes.

Official Documentation

These notes you are reading now are not exhaustive, but they will get you started! There is much more to Ohm. You will need to eventually check out the documentation.

Once you're feeling pretty good, you will find the Syntax Reference, API Reference, and Patterns and Pitfalls most useful.

Time to move to examples.

Example Grammars

Let's see Ohm in use with a few examples of our own. (There are more in the GitHub repo for Ohm.)

CLASSWORK

> We'll be going through these line by line, discussing not only Ohm specifics, but also a bit about language design in general and the intended meaning of each syntactic construct.

Bella

Bella is one of the example languages for this class. Here is its Ohm grammar:

bella.ohm

```
Bella {
  Program   = Statement+
  Statement = let id "=" Exp ";"                        -- vardec
            | function id Params "=" Exp ";"            -- fundec
            | Exp7_id "=" Exp ";"                       -- assign
            | print Exp ";"                             -- print
            | while Exp Block                           -- while
  Params    = "(" ListOf<id, ","> ")"
  Block     = "{" Statement* "}"
```

```
  Exp       = ("-" | "!") Exp7                              -- unary
            | Exp1 "?" Exp1 ":" Exp                         -- ternary
            | Exp1
  Exp1      = Exp1 "||" Exp2                                -- binary
            | Exp2
  Exp2      = Exp2 "&&" Exp3                                -- binary
            | Exp3
  Exp3      = Exp4 ("<="|"<"|"=="|"!="|">="|">") Exp4       -- binary
            | Exp4
  Exp4      = Exp4 ("+" | "-") Exp5                         -- binary
            | Exp5
  Exp5      = Exp5 ("*" | "/" | "%") Exp6                   -- binary
            | Exp6
  Exp6      = Exp7 "**" Exp6                                -- binary
            | Exp7
  Exp7      = num
            | true
            | false
            | id "(" ListOf<Exp, ","> ")"                   -- call
            | id                                            -- id
            | "(" Exp ")"                                   -- parens

  let       = "let" ~idchar
  function  = "function" ~idchar
  while     = "while" ~idchar
  true      = "true" ~idchar
  false     = "false" ~idchar
  print     = "print" ~idchar
  keyword   = let | function | while | true | false
  num       = digit+ ("." digit+)? (("E" | "e") ("+" | "-")? digit+)?
  id        = ~keyword letter idchar*
  idchar    = letter | digit | "_"
  space    += "//" (~"\n" any)*                             -- comment
}
```

Carlos

Carlos is another language for this class. It's a bit more sophisticated than Bella.

carlos.ohm

```
Carlos {
  Program     = Statement+

  Statement   = VarDecl
              | TypeDecl
              | FunDecl
              | Exp9 ("++" | "--") ";"                          --bump
              | Exp9 "=" Exp ";"                                --assign
              | Exp9_call ";"                                   --call
              | break ";"                                       --break
              | return Exp ";"                                  --return
              | return ";"                                      --shortreturn
              | IfStmt
              | LoopStmt

  VarDecl     = (let | const) id "=" Exp ";"
  TypeDecl    = struct id "{" Field* "}"
  Field       = id ":" Type
  FunDecl     = function id Params (":" Type)? Block
  Params      = "(" ListOf<Param, ","> ")"
  Param       = id ":" Type

  Type        = Type "?"                                        --optional
              | "[" Type "]"                                    --array
              | "(" ListOf<Type, ","> ")" "->" Type             --function
              | id                                              --id

  IfStmt      = if Exp Block else Block                         --long
              | if Exp Block else IfStmt                        --elsif
              | if Exp Block                                    --short
  LoopStmt    = while Exp Block                                 --while
              | repeat Exp Block                                --repeat
              | for id in Exp ("..." | "..<") Exp Block         --range
              | for id in Exp Block                             --collection
  Block       = "{" Statement* "}"
```

```
Exp         = Exp1 "?" Exp1 ":" Exp                          --conditional
            | Exp1
Exp1        = Exp1 "??" Exp2                                 --unwrapelse
            | Exp2
Exp2        = Exp3 ("||" Exp3)+                              --or
            | Exp3 ("&&" Exp3)+                              --and
            | Exp3
Exp3        = Exp4 ("|" Exp4)+                               --bitor
            | Exp4 ("^" Exp4)+                               --bitxor
            | Exp4 ("&" Exp4)+                               --bitand
            | Exp4
Exp4        = Exp5 ("<="|"<"|"=="|"!="|">="|">") Exp5        --compare
            | Exp5
Exp5        = Exp5 ("<<" | ">>") Exp6                        --shift
            | Exp6
Exp6        = Exp6 ("+" | "-") Exp7                          --add
            | Exp7
Exp7        = Exp7 ("*"| "/" | "%") Exp8                     --multiply
            | Exp8
Exp8        = Exp9 "**" Exp8                                 --power
            | Exp9
            | ("#" | "-" | "!" | some | random) Exp9         --unary
Exp9        = true ~mut
            | false ~mut
            | floatlit ~mut
            | intlit ~mut
            | no Type ~mut                                   --emptyopt
            | Exp9 ("(" | "?(") ListOf<Exp, ","> ")" ~mut    --call
            | Exp9 ("[" | "?[") Exp "]"                      --subscript
            | Exp9 ("." | "?.") id                           --member
            | stringlit ~mut
            | id                                             --id
            | Type_array "(" ")" ~mut                        --emptyarray
            | "[" NonemptyListOf<Exp, ","> "]" ~mut          --arrayexp
            | "(" Exp ")" ~mut                               --parens

intlit      = digit+
floatlit    = digit+ "." digit+ (("E" | "e") ("+" | "-")? digit+)?
stringlit   = "\"" char* "\""
```

```
  char        = ~control ~"\\" ~"\"" any
              | "\\" ("n" | "t" | "\"" | "\\")                --escape
              | "\\u{" hex hex? hex? hex? hex? hex? "}"       --codepoint
  control     = "\x00".."\x1f" | "\x80".."\x9f"
  hex         = hexDigit
  mut         = ~"==" "=" | "++" | "--"

  let         = "let" ~alnum
  const       = "const" ~alnum
  struct      = "struct" ~alnum
  function    = "function" ~alnum
  if          = "if" ~alnum
  else        = "else" ~alnum
  while       = "while" ~alnum
  repeat      = "repeat" ~alnum
  for         = "for" ~alnum
  in          = "in" ~alnum
  random      = "random" ~alnum
  break       = "break" ~alnum
  return      = "return" ~alnum
  some        = "some" ~alnum
  no          = "no" ~alnum
  true        = "true" ~alnum
  false       = "false" ~alnum
  keyword     = let | const | struct | function | if | else | while | repeat
              | for | in | break | return | some | no | random | true | false
  id          = ~keyword letter alnum*

  space      += "//" (~"\n" any)*                             --comment
}
```

Dax

Dax is a made up functional language.

dax.ohm

```
Dax {
  Program     = Exp
  Exp         = ("-" | "!") Primary                          --unary
              | Exp0 then Exp                                --then
              | Exp0
  Exp0        = Exp0 "|>" Exp1                               --pipe
              | Exp1
  Exp1        = Exp2 "?" Exp2 ":" Exp1                       --ternary
              | Exp2
  Exp2        = Exp3 ("&&" Exp3)+                            --and
              | Exp3 ("||" Exp3)+                            --or
              | Exp3
  Exp3        = Exp4 relop Exp4                              --compare
              | Exp4
  Exp4        = Exp4 ("+" | "-") Term                        --addsub
              | Term
  Term        = Term ("*" | "/" | "%") Factor                --muldiv
              | Factor
  Factor      = Primary "**" Factor                          --exponent
              | Primary
  Primary     = num
              | true
              | false
              | stringlit
              | let Dec (";" Dec)* in Exp kwend              --let
              | id Primary+                                  --id_call
              | id                                           --id
              | "(" Exp ")" Primary+                         --paren_call
              | "(" Exp ")"                                  --paren
              | "[" ListOf<Exp, ","> "]"                     --array
              | "{" id "=>" Exp "}" Primary+                 --lambda_call
              | "{" id "=>" Exp "}"                          --lambda
  Dec         = id "=" Exp
  relop       = "<=" | "<" | "==" | "!=" | ">=" | ">"
  then        = "then" ~idchar
  let         = "let" ~idchar
  in          = "in" ~idchar
  kwend       = "end" ~idchar
```

```
  true        = "true" ~idchar
  false       = "false" ~idchar
  keyword     = then | let | in | kwend | true | false
  id          = ~keyword letter idchar*
  idchar      = letter | digit | "_"
  num         = digit+ ("." digit+)? (("E" | "e") ("+" | "-")? digit+)?
  stringlit   = "\"" char* "\""
  char        = ~control ~"\\" ~"\"" any
              | "\\" ("n" | "t" | "\"" | "\\")              --escape
              | "\\u{" hex hex? hex? hex? hex? hex? "}"     --codepoint
  control     = "\x00".."\x1f" | "\x80".."\x9f"
  hex         = hexDigit
  space      += "//" (~"\n" any)* "\n"                     -- comment
}
```

A Java-like scripting language

Here is a small scripting language that is very Java-like in appearance, with a few simplifying features: (1) there are no classes, (2) the language needs little if any type inference, and (3) the standard library is both package-less and microscopic:

```
TinyJava {
  Program     =  Stmt+
  Stmt        =  Dec                          -- declaration
              |  Assignment ";"               -- assignment
              |  Call ";"                     -- call
              |  break ";"                    -- break
              |  return Exp? ";"              -- return
              |  print Args ";"               -- print
              |  if Exp Block
                 (else if Exp Block)*
                 (else Block)?                -- if
              |  while Exp Block              -- while
              |  for "("
                 (Type id "=" Exp)? ";"
                 Exp? ";"
                 Assignment? ")"
```

```
               Block                        -- for
Assignment  =  Increment                    -- increment
            |  Var "=" Exp                  -- plain
Increment   =  incop Var                    -- prefix
            |  Var incop                    -- postfix
Dec         =  TypeDec | VarDec | FunDec
TypeDec     =  struct id "{"
               (Type id ";")*
               "}"
Type        =  Type "[" "]"                 -- array
            |  primtype
            |  id
VarDec      =  Type id ("=" Exp)? ";"
FunDec      =  (Type | void) id "(" Params ")" Block
Params      =  (Type id ("," Type id)*)?
Block       =  "{" Stmt* "}"
Exp         =  NonemptyListOf<Exp1, "||">
Exp1        =  NonemptyListOf<Exp2, "&&">
Exp2        =  NonemptyListOf<Exp3, "|">
Exp3        =  NonemptyListOf<Exp4, "^">
Exp4        =  NonemptyListOf<Exp5, "&">
Exp5        =  Exp6 (relop Exp6)?
Exp6        =  NonemptyListOf<Exp7, shiftop>
Exp7        =  NonemptyListOf<Exp8, addop>
Exp8        =  NonemptyListOf<Exp9, mulop>
Exp9        =  prefixop? Exp10
Exp10       =  Literal
            |  Increment
            |  Var
            |  NewObject
            |  NewArray
            |  EmptyArray
            |  "(" Exp ")"                  -- parens
Literal     =  null
            |  true
            |  false
            |  floatlit
            |  intlit
            |  charlit
```

```
            |  stringlit
Var         =  Var "[" Exp "]"            -- subscript
            |  Var "." id                 -- select
            |  Call
            |  id
NewObject   =  new id "{" Args "}"
NewArray    =  new Type_array "{" Args "}"
EmptyArray  =  new Type "[" Exp "]"
Call        =  id "(" Args ")"
Args        =  ListOf<Exp, ",">

boolean     = "boolean" ~idchar
break       = "break" ~idchar
char        = "char" ~idchar
else        = "else" ~idchar
false       = "false" ~idchar
for         = "for" ~idchar
if          = "if" ~idchar
int         = "int" ~idchar
length      = "length" ~idchar
new         = "new" ~idchar
null        = "null" ~idchar
print       = "print" ~idchar
real        = "real" ~idchar
return      = "return" ~idchar
string      = "string" ~idchar
return      = "return" ~idchar
struct      = "struct" ~idchar
true        = "true" ~idchar
void        = "void" ~idchar
while       = "while" ~idchar

keyword     =  boolean | if | break | else | int | for | new
            |  return | char | struct | null | while | real
            | true | string | void | false | length | print
id          =  ~keyword letter idchar*
idchar      =  "_" | alnum
intlit      =  digit+
floatlit    =  digit+ "." digit+ (("E"|"e") ("+"|"-")? digit+)?
```

```
  char        =  escape
              |  ~"\\" ~"\"" ~"'" ~"\n" any
  h           =  hexDigit
  escape      =  "\\\\" | "\\\"" | "\\'" | "\\n" | "\\t" | hexseq
  hexseq      =  "\\" h h? h? h? h? h? h? h? ";"
  charlit     =  "'" (char | "\"") "'"
  stringlit   =  "\"" (char | "\'")* "\""
  addop       =  "+" | "-"
  relop       =  "<=" | "<" | "==" | "!=" | ">=" | ">"
  shiftop     =  "<<" | ">>"
  mulop       =  "*" | "/" | "%"
  prefixop    =  ~"--" "-" | "!" | "~" | char | int | string | length
  incop       =  "++" | "--"
  primtype    =  boolean | char | int | real | string

  space      :=  "\x20" | "\x09" | "\x0A" | "\x0D" | comment
  comment     =  "//" (~"\n" any)*
}
```

> ***Exercise**: Explain every line of this grammar. Note those things you find unconventional.*

JavaScript

Yes, there is a full ECMAScript grammar in Ohm out there.

Brainfuck

Brainfuck has a pretty trivial grammar. There are eight one-character instructions, and every other character is ignored.

```
Brainfuck {
  Program     = instruction+
  instruction = "[" instruction* "]"                -- loop
              | "," | "." | "+" | "-" | "<" | ">"
```

```
  space       = ~instruction any
}
```

***Exercise**: Later, after learning about semantics objects in Ohm, write a semantics for this grammar to create a compiler that translates Brainfuck into C. See the [Wikipedia article] for hints.*

Example Applications

Ohm can do so much more—so much more—than just generate ASTs. You can do just about anything in the semantics (parsing functions). Here are a few case studies:

Infix to Postfix Conversion

Here is a language of infix arithmetic expressions over numbers and identifiers. From lowest to highest the precedences are: (1) the left-associative additive operators, (2) left-associative multiplicative operators, (3) the prefix unary negation operator, (4) right-associative the exponentiation operator.

infix-to-postfix.mjs

```
import * as ohm from 'ohm-js'

const grammar = ohm.grammar(`Infix {
  Program = Exp
  Exp     = Exp ("+" | "-") Term     --binary
          | Term
  Term    = Term ("*" | "/") Factor  --binary
          | Factor
  Factor  = "-" Factor               --negate
          | Primary "**" Factor      --binary
          | Primary
  Primary = "(" Exp ")"              --parens
          | digit+                   --number
          | letter alnum*            --identifier
}`)
```

```
const semantics = grammar.createSemantics().addOperation('post', {
  Program(body) {
    return body.post()
  },
  Exp_binary(left, op, right) {
    return `${left.post()} ${right.post()} ${op.sourceString}`
  },
  Term_binary(left, op, right) {
    return `${left.post()} ${right.post()} ${op.sourceString}`
  },
  Factor_binary(left, op, right) {
    return `${left.post()} ${right.post()} ${op.sourceString}`
  },
  Factor_negate(_op, operand) {
    return `${operand.post()} ~`
  },
  Primary_parens(_open, expression, _close) {
    return expression.post()
  },
  Primary_number(_chars) {
    return this.sourceString
  },
  Primary_identifier(_first, _rest) {
    return this.sourceString
  },
})

export function toPostfix(infix) {
  let match = grammar.match(infix)
  if (!match.succeeded()) {
    throw new Error(match.message)
  }
  return semantics(match).post()
}
```

Unit tests:

infix-to-postfix-test.mjs

```
import { describe, it } from "node:test"
import assert from "assert/strict"
import { toPostfix } from "../infix-to-postfix.mjs"

describe("When converting infix to postfix", () => {
  it("works", () => {
    assert.equal(toPostfix("1 + 2"), "1 2 +")
    assert.equal(toPostfix("1 + 2 - 3"), "1 2 + 3 -")
    assert.equal(toPostfix("1 ** 2 ** 3"), "1 2 3 ** **")
    assert.equal(toPostfix("-1 ** 2 ** 3"), "1 2 3 ** ** ~")
  })
})
```

Run the tests with a command such as `node --test test/infix-to-postfix-test.mjs`

> ***Exercise**: Write a command line application that does the infix to postfix conversion, using the module above.*

Evaluating and Differentiating Polynomials 😍

This fun script illustrates multiple semantics for the same grammar.

polynomial.mjs

```
import * as ohm from 'ohm-js'

const grammar = ohm.grammar(`Polynomial {
  Poly
    = Poly "+" term              -- add
    | Poly "-" term              -- subtract
    | "-" Poly                   -- negate
    | term
  term
    = coefficient "x^" exponent  -- coeff_var_exp
    | coefficient "x"            -- coeff_var
    | coefficient                -- coeff
    | "x^" exponent              -- var_exp
```

```
    | "x"                          -- var
  coefficient
    = digit+ ("." digit+)?
  exponent
    = "-"? digit+
}`)

/* eslint-disable no-unused-vars */
const semantics = grammar
  .createSemantics()
  .addOperation('deriv', {
    Poly_add(p, op, t) {
      return `${p.deriv()}+${t.deriv()}`
    },
    Poly_subtract(p, op, t) {
      return `${p.deriv()}-${t.deriv()}`
    },
    Poly_negate(_, p) {
      return `-${p.deriv()}`
    },
    term_coeff_var_exp(c, _, e) {
      return `${e.value * c.value}x^${e.value - 1}`
    },
    term_coeff_var(c, _) {
      return `${c.value}`
    },
    term_coeff(c) {
      return '0'
    },
    term_var_exp(_, e) {
      return `${e.value}x^${e.value - 1}`
    },
    term_var(_) {
      return '1'
    },
  })
  .addOperation('eval', {
    Poly_add(p, op, t) {
      return x => p.eval()(x) + t.eval()(x)
```

```
    },
    Poly_subtract(p, op, t) {
      return x => p.eval()(x) - t.eval()(x)
    },
    Poly_negate(_, p) {
      return x => -p.eval()(x)
    },
    term_coeff_var_exp(c, _, e) {
      return x => c.value * x ** e.value
    },
    term_coeff_var(c, _) {
      return x => c.value * x
    },
    term_coeff(c) {
      return x => c.value
    },
    term_var_exp(_, e) {
      return x => x ** e.value
    },
    term_var(_) {
      return x => x
    },
  })
  .addAttribute('value', {
    coefficient(whole, dot, fraction) {
      return +this.sourceString
    },
    exponent(sign, magnitude) {
      return +this.sourceString
    },
  })

export function derivative(poly) {
  const match = grammar.match(poly)
  if (!match.succeeded()) {
    throw new Error(match.message)
  }
  return semantics(match).deriv()
    .replace(/\-\-/g, '+')
```

```
    .replace(/\+\-/g, '-')
    .replace(/^\+/, '')
    .replace(/\+0(?!\d)/, '')
    .replace(/\^1(?!\d)$/, '')
}

export function evaluate(poly, x) {
  const match = grammar.match(poly)
  if (!match.succeeded()) {
    throw new Error(match.message)
  }
  return semantics(match).eval()(x)
}
```

With unit tests even!

poly-test.mjs

```
import { describe, it } from "node:test"
import assert from "assert/strict"
import { derivative, evaluate } from "../polynomial.mjs"

describe("The differentiator", () => {
  it("detects malformed polynomials", () => {
    assert.throws(() => derivative("2y"))
    assert.throws(() => derivative("blah"))
    assert.throws(() => derivative("2x*6"))
  })
  it("correctly differentiates single-term polynomials", () => {
    assert.equal(derivative("4"), "0")
    assert.equal(derivative("2238"), "0")
    assert.equal(derivative("x"), "1")
    assert.equal(derivative("4x"), "4")
    assert.equal(derivative("x^5"), "5x^4")
    assert.equal(derivative("2x^-4"), "-8x^-5")
  })
  it("correctly differentiates multi-term polynomials", () => {
    assert.equal(derivative("-4"), "-0")
    assert.equal(derivative("-2238"), "-0")
    assert.equal(derivative("-x"), "-1")
```

```
    assert.equal(derivative("-x^5"), "-5x^4")
    assert.equal(derivative("-x^-5"), "5x^-6")
    assert.equal(derivative("2x^-4    + 7x^2"), "-8x^-5+14x")
    assert.equal(derivative("   2x^-4- 7x^20"), "-8x^-5-140x^19")
    assert.equal(derivative("2x^-4       +7x^-2"), "-8x^-5-14x^-3")
  })
})

describe("The evaluator", () => {
  it("detects malformed polynomials", () => {
    assert.throws(() => evaluate("2y"))
    assert.throws(() => evaluate("blah"))
    assert.throws(() => evaluate("2x*6"))
  })
  it("correctly evaluates single-term polynomials", () => {
    assert.equal(evaluate("4", 5), 4)
    assert.equal(evaluate("2238", 19), 2238)
    assert.equal(evaluate("x", 3), 3)
    assert.equal(evaluate("4x", 10), 40)
    assert.equal(evaluate("x^5", 2), 32)
    assert.equal(evaluate("32x^-4", 2), 2)
  })
  it("correctly evaluates complex polynomials", () => {
    assert.equal(evaluate("-4", 8), -4)
    assert.equal(evaluate("-2238", 11), -2238)
    assert.equal(evaluate("-x", 5), -5)
    assert.equal(evaluate("-x^5", 3), -243)
    assert.equal(evaluate("-x^-3", 2), -0.125)
    assert.equal(evaluate("2x^8    + 7x^2", 2), 540)
    assert.equal(evaluate("   2x^3- 7x^1", 4), 100)
    assert.equal(evaluate("2x^3       +7x^1", 4), 156)
  })
})
```

CLASSWORK

We'll build a command line application allowing differentiation and evaluation of polynomials. Should be fun!

More Examples

The Ohm repository on GitHub has a nice list of Examples.

Recall Practice

Here are some questions useful for your spaced repetition learning. Many of the answers are not found on this page. Some will have popped up in lecture. Others will require you to do your own research.

What can you do with Ohm? HIDE ANSWER

(1) Define the syntax and semantics of a language, and (2) easily write parsers, interpreters, compilers, and more.

Why is the Ohm Editor useful? HIDE ANSWER

It allows for an interactive programming language design by providing immediate feedback as you author the grammar.

What is the difference between `x` and `"x"` in Ohm? HIDE ANSWER

`x` is a rule, `"x"` is a literal.

What is the difference between lexical and syntax rules in Ohm? HIDE ANSWER

Lexical rules are for tokens, syntax rules are for phrases that are allowed to implicitly have spaces between components.

What does the parsing expression `e1 | e2` mean? HIDE ANSWER

Try to match `e1`, and if that fails, try to match `e2`.

What does the parsing expression `e*` mean? HIDE ANSWER

Match `e` zero or more times.

What does the parsing expression `e+` mean? HIDE ANSWER

Match `e` one or more times.

What does the parsing expression `e?` mean? HIDE ANSWER

Match `e` zero or one times.

What does the parsing expression `&e` mean? HIDE ANSWER

Match `e` without consuming it (positive lookahead).

What does the parsing expression `~e` mean? HIDE ANSWER

Succeed if `e` doesn't match, without consuming anything (negative lookahead).

What is the built-in rule that matches the end of the input in Ohm? HIDE ANSWER

```
end
```

How do you represent "any character except a newline" in Ohm? HIDE ANSWER

```
~"\n" any
```

Why does Ohm require each branch of an alternation to have the same arity? HIDE ANSWER

So that parsing actions can take a fixed number of parameters.

What cool shorthand does Ohm allow so that you don't have to explicitly created separate related rules to make sure arity restrictions are followed? HIDE ANSWER

You can use `--label` at the end of alternatives.

How does the `match` method on an Ohm grammar object work? HIDE ANSWER

It returns a `MatchResult` object that wraps a parse tree if the match succeeded.

What are the main methods and properties of an Ohm MatchResult? HIDE ANSWER

`succeeded()`, `failed()`, `message`, `shortMessage`.

How do you obtain a semantics object for a grammar *g* in Ohm? HIDE ANSWER

```
g.createSemantics()
```

What is the difference between operations and attributes in an Ohm semantics object? HIDE ANSWER

Operations are functions that are called every time, attributes are memoized and called with property syntax.

What are the three kinds of parse nodes in Ohm? HIDE ANSWER

Rule application nodes, iteration nodes, terminal nodes.

If you call an action method on a terminal node, but there is no associated method in the action dictionary, what happens? HIDE ANSWER

The special method `_terminal` is called, which by default returns `this.sourceString`.

If you call an action method on an iteration node, but there is no associated method in the action dictionary, what happens? HIDE ANSWER

You will get an error, unless you have properly defined an `_iter` method in the dictionary.

Is it a best practice to fall back to the `_iter` method? HIDE ANSWER

No, it is better to write `myNode.children.map(c => c.rep())` directly each time you have to process an iteration node.

Summary

We've covered:

- ✅ A good deal about Ohm
- ✅ Where to find Ohm's documentation
- ✅ Example Ohm Grammars
- ✅ Examples of semantic processing in Ohm